第10章: デプロイと総まとめ

> おめでとうございます！最終章です。この章では、ローカル（自分のPC）で動いているアプリを**インターネット上に公開**して、誰でもアクセスできるようにします。

この章で行うこと

作業	内容	所要時間の目安
ビルド	Build：開発用コードを本番環境用に最適化する。ファイルサイズの圧縮やコードの最適化が行われる	5分
デプロイ	Deploy：最適化したコードをサーバーに配置して公開する。「あなたのアプリにURLが付いて、世界中からアクセスできる状態になる」こと	10〜15分
本番設定	Supabaseのセキュリティ設定の確認と調整	10分
振り返り	この教材で学んだことの総まとめ	10分

> **デプロイとは？**「デプロイ（Deploy）」は英語で「配置する／配備する」という意味の単語です。プログラミングの世界では、**自分のPCで動いているアプリを、インターネット上のサーバーに置いて、URLを叩けば世界中の誰でも使える状態にする作業**を指します。自分のPCで動いているアプリは、他の人からはアクセスできません。デプロイをすると、`https://～` のURLが発行され、スマホからでも、海外からでもアクセスできるようになります。レストランで例えると、自宅で試作した料理を実店舗で提供開始する段階です。

この章では、**Vercel**（ヴァーセル：Next.js を開発している会社が提供するホスティングサービス。GitHub と連携するだけで自動デプロイできる）を使ってデプロイします。そして、この教材全体を通じて学んだことを振り返り、今後の学習ロードマップ（次に何を学ぶべきかの道筋）を示します。

> **ホスティング（Hosting）とは？**「Host（ホスト）」は「招き入れる人」という意味です。**自分が作ったWebアプリやファイルを、24時間動いているサーバー（コンピューター）に置いて、世界中からアクセスできるようにするサービス**を「ホスティングサービス」と呼びます。Vercel、Netlify、AWS などが代表例です。自分のPCはずっと電源を入れっぱなしにできないので、専門の会社のサーバーを借りる、というイメージです。

> **サーバレス（Serverless）とは？** 直訳すると「サーバーがない」ですが、**「サーバーがない」のではなく「サーバーの存在を意識しなくていい」**という意味です。従来は自分でサーバーを用意・管理する必要がありましたが、サーバレスでは「コードを置くだけ」で、必要なときに自動でサーバーが動き、終わったら止まります。料金もアクセスがあった分だけ。Vercel はこのサーバレスの仕組みでアプリを動かしています。

> **Edge Function（エッジ関数）とは？** 「Edge」は「端っこ」という意味で、ここでは**世界中に散らばっているサーバー（ユーザーに近い場所）**を指します。Edge Function は、そのユーザーに一番近いサーバーで実行されるプログラムのことで、応答が速いのが特徴です。Vercel ではこの仕組みでページを高速配信しています。

この章のゴールと地図

この章を読み終えると、次のことができるようになります。

- 自分のPCで `npm run build` を通し、よくあるビルドエラーを自分で読んで直せる
- GitHub にあるコードを Vercel に取り込み、環境変数を登録して、`https://～.vercel.app` で公開できる
- 「手元では動くのに本番では動かない」ときに、どのログを見ればよいか判断できる
- 不具合が出たときに、ひとつ前のバージョンへ戻す（ロールバックする）方法を知っている

デプロイは、次の6つの「駅」を順番に進む旅だと考えてください。図の下の「§」は、その駅を説明している節の番号です。

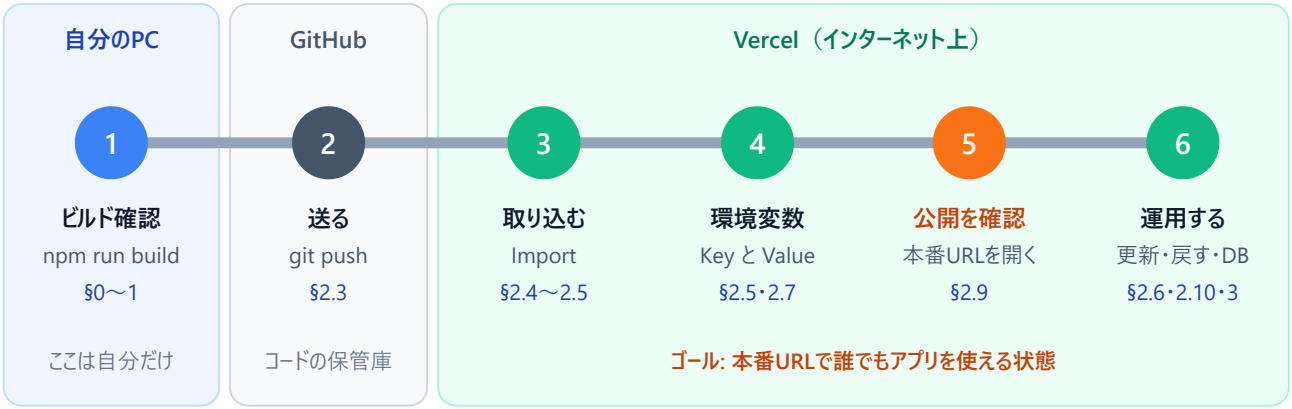


図10-1: この章の地図（6つの駅を順番に進むと、アプリが世界に公開される）

> **ヒント:** デプロイでいちばん多い失敗は、「手元で動いていたから大丈夫」と思って、ビルドを確かめずに本番へ出してしまうことです。1番目の駅（ビルド確認）を飛ばさないだけで、トラブルの大半は防げます。

目次

0. 前提知識: ビルド／デプロイ／環境変数とは

1. デプロイの準備
2. Vercel へのデプロイ
3. Supabase の本番設定
4. パフォーマンス最適化
5. 学習の振り返り
6. 次のステップ（発展学習）
7. 推奨学習リソース
8. 付録: 全ファイルの完成版一覧
9. おわりに

> **ヒント:** 目次の前に「この章のゴールと地図」があります。また、「8. 付録」と「9. おわりに」の間に「練習問題（解答・解説つき）」があります。

---

0. 前提知識: ビルド／デプロイ／環境変数とは

0.1 ビルド（Build）とは

「**書いたコードを、本番で実行できる形に変換する**」処理です。`npm run build` で実行します。具体的には:

1. **TypeScript → JavaScript** に変換（ブラウザは TS を読めない）
2. **JSX → 通常のJS関数呼び出し** に変換
3. **ファイルを圧縮**（不要な空白・コメント削除、変数名を短縮）
4. **ファイルを結合（バンドル）**してリクエスト数を削減
5. **画像やフォントを最適化**

▼ ビルド実行例:

コピー

```
# $ はターミナルの入力待ち記号（自分でタイプする必要はない）
# npm = Node.js に付属するパッケージマネージャ
# run = package.json の scripts に書かれたコマンドを実行する命令
# build = package.json で定義された「ビルド用」のスクリプト名（中身は "next build"）
$ npm run build
```

▼ 期待される出力（抜粋。Next.js 15 での表示例）:

コピー

```
> next build

   ▲ Next.js 15.0.0
   - Environments: .env.local

   Creating an optimized production build ...
 ✓ Compiled successfully in 12.3s
   Linting and checking validity of types
 ✓ Generating static pages (5/5)
   Finalizing page optimization

Route (app)                              Size     First Load JS
┌ ƒ /                                    1.32 kB        92.5 kB
├ ○ /books/new                           2.45 kB        93.6 kB
├ ƒ /books/[id]                          1.78 kB        92.9 kB
└ ƒ /books/[id]/edit                     2.10 kB        93.2 kB

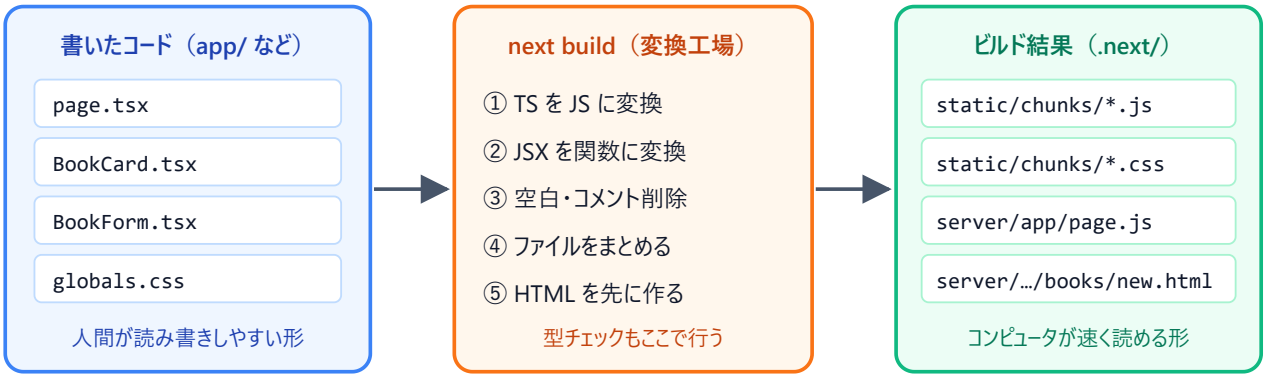
○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand
```

▼ 何が起きた？

- すべてのページが正常にビルドされた
- 各ページのファイルサイズが表示される
- `○` は静的ページ（あらかじめHTMLが用意されている）、`ƒ` はリクエストごとにサーバーで生成されるページ（古い Next.js では `λ` と表示されていました）
- `First Load JS` は「最初にユーザーのブラウザがダウンロードする JavaScript の量」。小さいほど読み込みが速い

> **Next.js 16 では表示が少し変わります:** 2026年10月時点の最新版（16.x）では `Size` と `First Load JS` の列が表示されなくなり、`Linting and checking validity of types` の行は `Running TypeScript ...` に変わりました。16.4.1 で実際に実行した出力は「1.2 ビルドの実行」で詳しく読みます。

ビルドの前と後でファイルがどう変わるのかを絵にすると、次のようになります。



ファイル名の英数字（ハッシュ）は、中身が変わるたびに変わります

図10-2: ビルド（next build）の前と後。app/ や components/ のコードが、.next/ フォルダの「配る用のファイル」に変換される

> **ビルド失敗が出たら？**: TypeScriptの型エラーや lint エラーが残っているとビルドできません。エラーメッセージのファイル名と行番号を見て修正します。**ローカルでビルドが通ってからデプロイする習慣**をつけると安全です。本番のVercel上でビルドに失敗すると、せっかく `git push` しても新しいバージョンが反映されないため、必ず手元で `npm run build` を1回通してからプッシュしましょう。

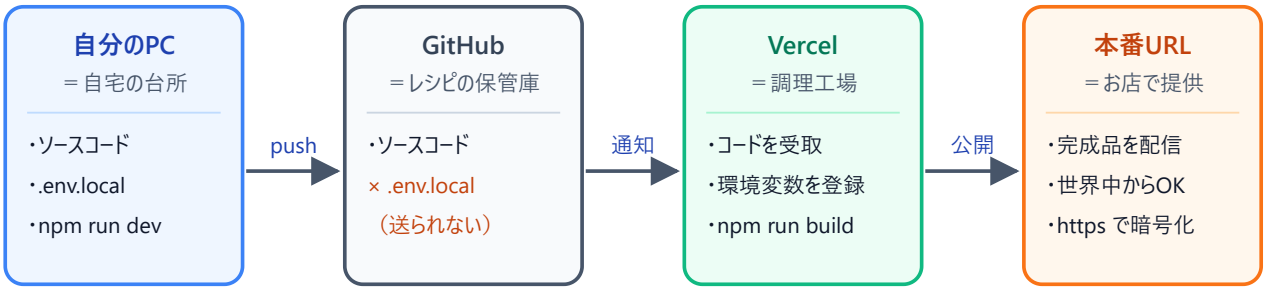
0.2 デプロイ（Deploy）とは

ビルドの成果物を**インターネット上のサーバーに配置**して、誰でも URL でアクセスできるようにする作業です。本書では **Vercel** に GitHub 経由で自動デプロイします。

コピー

```
[あなたのPC]                    ← ここでコードを書く
   ↓ git push                    ← GitHub にコードを送る命令
[GitHub]                        ← コードの保管庫（バージョン管理サービス）
   ↓ Webhookで自動通知            ← Webhook = ある出来事が起きたら別のサービスに自動で知らせる仕組み
[Vercel]                        ← Next.js 公式のホスティングサービス
   ↓ npm run build を実行        ← Vercel が自動でビルドしてくれる
[CDN（世界中のサーバー）]          ← CDN = Content Delivery Network、世界中の中継サーバー網
   ↓
あなたのアプリ: https://your-app.vercel.app   ← この URL で公開される
```

同じ流れを、料理店にたとえて絵にすると次のようになります。特に「`.env.local` は GitHub に送られない」という点は、この章の後半で何度も出てくる大事なポイントです。



ポイント: .env.local は GitHub に送られません。だから Vercel にも同じ環境変数を登録する必要があります（§2.5 Step 3）。忘れると「本番だけ動かない」原因になります。

図10-3: ローカル → GitHub → Vercel → 本番URL の全体の流れ（料理店にたとえると）

CI/CD（シーアイ・シーディー）とは？「Continuous Integration（継続的インテグレーション）」と「Continuous Delivery / Deployment（継続的デリバリー／デプロイ）」の略。**コードを変更するたびに、テスト・ビルド・デプロイを自動で行う仕組み**のことです。Vercel + GitHub の組み合わせも、簡単な CI/CD の一種で、`git push` するだけで自動的にビルドからデプロイまで進みます。手作業のミスを減らせるのが大きなメリットです。

プレビュー環境（Preview Environment）とは？ 本番に反映する前に、**変更内容を試せる「お試し版」のURL**のこと。Vercel では、Pull Request（変更案の提案）を作るたびに自動で `https://your-app-git-feature-xxx.vercel.app` のような専用URLが生成され、本番のデータには影響を与えずに動作確認ができます。

ロールバック（Rollback）とは？「Roll（巻く）」+「Back（戻す）」で、**「変更を取り消して前の状態に戻す」**こと。例えば、新しくデプロイしたバージョンに不具合があった場合、Vercel のダッシュボードからボタン1つで以前動いていたバージョンに戻せます。これがあるので、安心して新しいバージョンを試せます。

ドメイン／DNS／SSL とは？

- **ドメイン（Domain）**：`google.com` や `your-app.vercel.app` のような、人間が覚えやすいインターネット上の住所のこと。本来サーバーは `192.168.x.x` のような数字（IPアドレス）で識別されますが、覚えるのが大変なので、文字列の住所を割り当てます。
- **DNS（Domain Name System）**：ドメイン名と IPアドレスを変換してくれる「インターネットの電話帳」。「`your-app.vercel.app` ってどこ？」と聞くと、「IPアドレスは○○です」と教えてくれます。
- **SSL/TLS**: 通信を暗号化する技術。「Secure Sockets Layer」「Transport Layer Security」の略。URL が `http://` ではなく `https://` で始まっていれば SSL/TLS で暗号化されている証拠で、鍵マークがブラウザに表示されます。Vercel では自動的に有効になっています。

0.3 環境変数（Environment Variables）とは

「コードに直接書きたくない設定値」を別の場所で管理する仕組みです。代表例：

変数名（例）	用途	機密性
`NEXT_PUBLIC_SUPABASE_URL`	Supabase のURL	公開可（ブラウザに送られる）
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase の公開用キー（値は Publishable key `sb_publishable_...`。旧 anon key でも動く）	公開可
`SUPABASE_SERVICE_ROLE_KEY`	Supabase の管理者キー（Secret key `sb_secret_...`。旧 service_role key）	**絶対秘密**（サーバー専用。本書のアプリでは使わない）

ローカル: `.env.local` ファイルに書く（`.gitignore` でGit管理から外れている）

コピー

```
# .env.local（このファイルはGitHubに上げない！）
# 「#」で始まる行はコメント。実行時には無視される
# 「変数名=値」の形式で1行に1つの環境変数を書く（=の左右にスペースを入れない）

# NEXT_PUBLIC_ プレフィックス付きなのでブラウザにも値が送られる。URL は公開しても問題ない情報
NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co

# 公開用キー（Publishable key）。変数名は ANON_KEY のまま、値に sb_publishable_ で始まるキーを入れる（第5章）
# 公開前提のキーなので、RLS（Row Level Security）でデータを守ることが必須
NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_xxxxxxxxxxxxxxxx
```

本番（Vercel）: Vercel ダッシュボードでプロジェクトを開き、**Settings → Environment Variables** で同じ値を設定する（手順は 2.5 と 2.7）。

2つの「読み込み元」の関係を図にすると、次のようになります。手元と本番では、環境変数を読む場所がまったく別だという点を押さえてください。

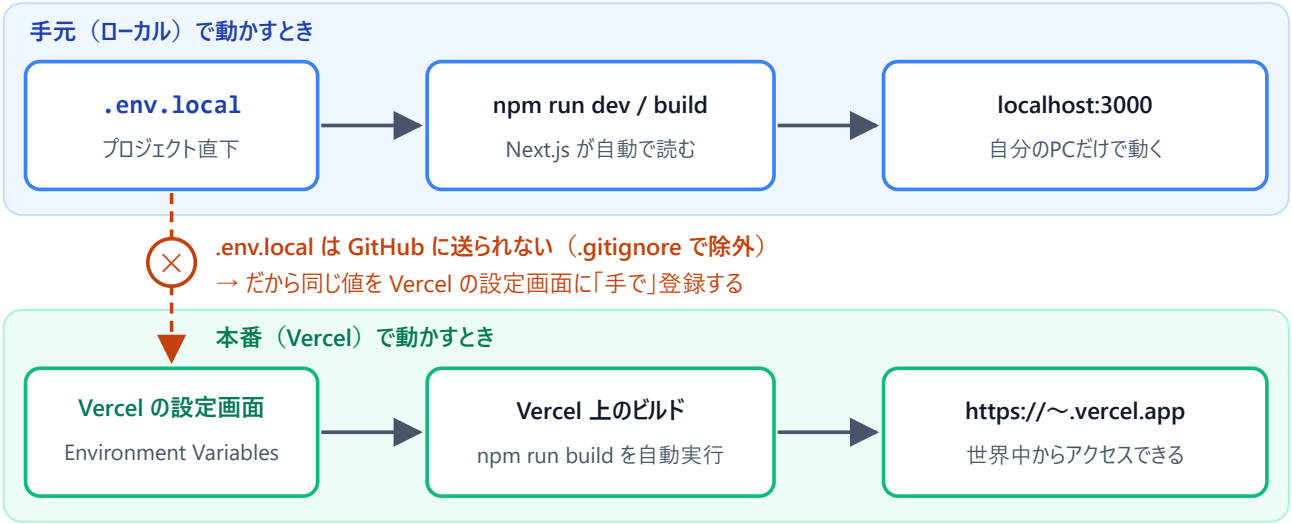


図10-4: 環境変数はどこから読まれるか（ローカルは .env.local、本番は Vercel の設定画面）

> **`NEXT_PUBLIC_` の意味:** 接頭辞 `NEXT_PUBLIC_` を付けた環境変数は**ブラウザ側にも送られる**（=世界に公開される）。付けない変数はサーバーでしか参照できないので、シークレットには `NEXT_PUBLIC_` を付けてはいけない。Next.js は、ビルド時に `NEXT_PUBLIC_` 付きの環境変数を JavaScript ファイルの中に埋め込みます。そのためビルド後にブラウザの開発者ツールでソースを開くと、その値が文字列として見えてしまいます。逆に `NEXT_PUBLIC_` が付いていない変数は、サーバー上でしか参照できず、ブラウザには出力されません。

実験1: `NEXT_PUBLIC_` の値は本当に JS に書き込まれるのか

「ブラウザに送られる」「JS に埋め込まれる」と言われても、目で見ないと実感がわきません。小さな実験で確かめてみましょう。書籍管理アプリのプロジェクトで試してかまいません（実験が終わったら、作ったファイルと追加した2行は削除してください）。

> **▼ このコードがやること（先に日本語で）:** 「公開してよい値」と「秘密の値」を1つずつ `.env.local` に書き足し、その2つを画面に表示しようとする小さな部品とページを作ります。ビルドしたあと、ブラウザに配られる JS ファイルの中を検索して、どちらの値が入っているかを確かめます。

コピー

```
# .env.local の末尾に、実験用の2行を書き足す（本物の秘密は書かないこと）
NEXT_PUBLIC_APP_NAME=MyBookShelf-PUBLIC-123
SECRET_MESSAGE=TopSecret-XYZ-789
```

コピー

```
// components/EnvViewer.tsx（実験用の部品）
"use client";

export default function EnvViewer() {
  // (1) NEXT_PUBLIC_ 付き: ビルド時に値が埋め込まれる
  const publicName = process.env.NEXT_PUBLIC_APP_NAME;
  // (2) NEXT_PUBLIC_ なし: ブラウザには届かない
  const secret = process.env.SECRET_MESSAGE;

  return (
    <ul>
      <li>公開してよい値: {publicName ?? "(undefined)"}</li>
      <li>秘密の値: {secret ?? "(undefined)"}</li>
    </ul>
  );
}
```

コピー

```
// app/env-check/page.tsx（実験用のページ。/env-check で開ける）
import EnvViewer from "@/components/EnvViewer";

export default function EnvCheckPage() {
  return (
    <main className="p-6">
      <h1>環境変数チェック</h1>
      <EnvViewer />
    </main>
  );
}
```

コピー

```
# 1) ビルドする
npm run build
# 2) ブラウザに配られる JS（.next\static\chunks）の中から、公開してよい値を探す
Select-String -Path .next\static\chunks\*.js -Pattern "MyBookShelf-PUBLIC-123" -List | Select-Object Filename
# 3) 同じように、秘密の値を探す
Select-String -Path .next\static\chunks\*.js -Pattern "TopSecret-XYZ-789" -List | Select-Object Filename
```

▼ **実行結果**（Windows の PowerShell。ファイル名の英数字は環境ごとに変わります）

コピー

```
PS> Select-String -Path .next\static\chunks\*.js -Pattern "MyBookShelf-PUBLIC-123" -List | Select-Object Filename

Filename
--------
0vu48qaqft2br.js

PS> Select-String -Path .next\static\chunks\*.js -Pattern "TopSecret-XYZ-789" -List | Select-Object Filename
PS>
```

公開してよい値は見つかり、秘密の値は何も表示されませんでした（＝どのファイルにも入っていない）。

> **ヒント:** Mac のターミナルでは `grep -rl "MyBookShelf-PUBLIC-123" .next/static` で同じことを確かめられます（見つかったファイルの場所が表示されます）。

▼ コードを1つずつ分解して解説

解説1: Client Component の中で環境変数を読む

コピー

```
const publicName = process.env.NEXT_PUBLIC_APP_NAME;
const secret = process.env.SECRET_MESSAGE;
```

- この部品は `"use client"` 付きの Client Component（ブラウザでも動く部品）なので、ここに書いた `process.env.〜` は、ビルドのときに「ブラウザ用の JS」へ変換されます。
- `NEXT_PUBLIC_` 付きの方は、ビルドの瞬間に `"MyBookShelf-PUBLIC-123"` という**値そのもの**に書き換えられます。付いていない方は書き換えられず、ブラウザでは `undefined`（値が無い）になります。

解説2: `Select-String` で JS ファイルの中を検索する

コピー

```
Select-String -Path .next\static\chunks\*.js -Pattern "MyBookShelf-PUBLIC-123" -List | Select-Object Filename
```

- `Select-String` は PowerShell の「文字列を探す」コマンドです。`-Path` で探す場所（`.next\static\chunks` の中の `.js` ファイル全部）を、`-Pattern` で探す文字列を指定します。
- `-List` は「1ファイルにつき最初の1件だけ」、`| Select-Object Filename` は「見つかったファイル名だけを表示する」という意味です。圧縮された JS は1行がとても長いので、中身をそのまま表示すると画面が埋まってしまいます。

> **用語:** `.next\static\chunks` … ビルドで作られた「ブラウザに配る JS」が置かれるフォルダ。ここにあるファイルは、本番ではサイトを開いた人なら誰でもダウンロードできます。

解説3: 結果の読み方

- 公開してよい値は `0vu48qaqft2br.js` の中から見つかりました。つまり、**本番サイトを開いた人は誰でも、この値を読める**ということです。Supabase の URL や公開用キーが `NEXT_PUBLIC_` 付きで問題ないのは、「誰に見られても困らない値」だからです。
- 秘密の値はどこからも見つかりませんでした。`NEXT_PUBLIC_` を付けなければ、ブラウザ用の JS には入りません。

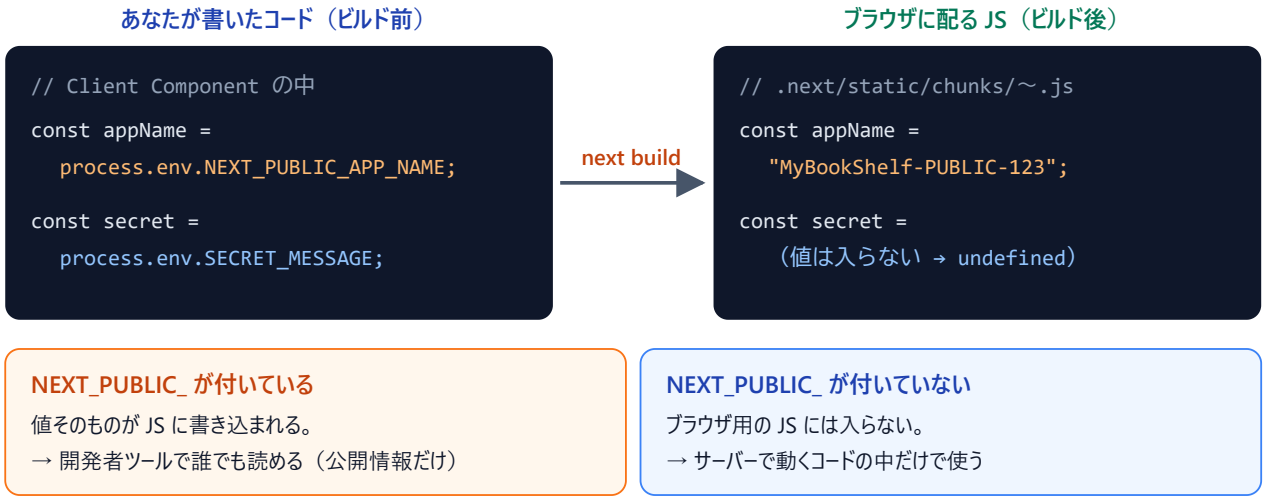


図10-5: NEXT_PUBLIC_ 付きの環境変数は、ビルドの瞬間に「値」へ置き換わって JS に焼き付けられる

> **つまずきポイント:**「ブラウザ用の JS に入らないなら、Client Component で秘密の値を読んでも安全」……ではありません。上の実験のページを本番モードで動かし、サーバーから届く HTML を調べると、秘密の値がそのまま入っています。

コピー

```
# 本番モードで起動する（-p 3001 はポート番号の指定。1.2.1 参照）
npm run start -- -p 3001
# 別の PowerShell を開き、サーバーが返す HTML の中から秘密の値を探す
(curl.exe -s http://localhost:3001/env-check | Select-String "TopSecret-XYZ-789").Matches.Value
```

▼ 実行結果

コピー

```
PS> (curl.exe -s http://localhost:3001/env-check | Select-String "TopSecret-XYZ-789").Matches.Value
TopSecret-XYZ-789
```

ブラウザの画面には次のように表示されているのに、です。

コピー

```
公開してよい値: MyBookShelf-PUBLIC-123
秘密の値: (undefined)
```

Next.js は Client Component も、まずサーバー側（このページではビルドのとき）で HTML を作ってから送ります。サーバーでは秘密の環境変数も読めるので、そのときの値が HTML に書き込まれてしまうのです。ブラウザで JS が動くと `(undefined)` に置き換わるため画面からは消えますが、右クリック →「ページのソースを表示」を開けば誰でも読めます。

NG: 上の `EnvViewer.tsx` のように、`"use client"` の部品の中で `NEXT_PUBLIC_` の付いていない秘密の値を読む。

OK: 秘密の値は Server Component（`"use client"` を書かないファイル）や API の中だけで使い、画面には「結果」だけを出す。

コピー

```
// app/env-check-ok/page.tsx（OK: "use client" を書かない＝ Server Component）
export default function EnvCheckOkPage() {
  // Server Component の中なら、秘密の値を読んでもブラウザ用の JS には入らない
  const hasSecret = Boolean(process.env.SECRET_MESSAGE);

  return (
    <main className="p-6">
      {/* 値そのものではなく「設定されているか」だけを表示する */}
      <p>秘密の値は設定済み？ {hasSecret ? "はい" : "いいえ"}</p>
    </main>
  );
}
```

▼ **実行結果**（HTML の中に秘密の値は見つからない。画面には「秘密の値は設定済み？ はい」と表示される）

コピー

```
PS> (curl.exe -s http://localhost:3001/env-check-ok | Select-String "TopSecret-XYZ-789").Matches.Value
PS>
```

> **つまずきポイント:** `NEXT_PUBLIC_` 付きでも、コードに **`process.env.NEXT_PUBLIC_〜`** と「そのまま」書かないと値が埋め込まれません。環境変数の名前を引数で受け取る「便利関数」を作ると、ブラウザでは `undefined` になります。

NG: 名前を引数で渡して `process.env[name]` で読む

コピー

```
"use client";

// 環境変数の名前を受け取って値を返す「便利関数」（のつもり）
function readEnv(name: string) {
  return process.env[name];
}

export default function EnvViewer() {
  // OK: ドット記法でそのまま書く（ビルド時に値へ置き換わる）
  const direct = process.env.NEXT_PUBLIC_APP_NAME;
  // NG: 名前を引数で渡して [ ] で読む（置き換えが起きない）
  const viaFunction = readEnv("NEXT_PUBLIC_APP_NAME");

  return (
    <ul>
      <li>ドット記法: {direct ?? "(undefined)"}</li>
      <li>関数経由: {viaFunction ?? "(undefined)"}</li>
    </ul>
  );
}
```

▼ **実行結果**（ビルドして本番モードで `/env-check` を開いたときの画面）

コピー

```
ドット記法: MyBookShelf-PUBLIC-123
関数経由: (undefined)
```

Next.js がビルドのときに値へ置き換えるのは、ソースコードに `process.env.NEXT_PUBLIC_APP_NAME` と**文字どおり書かれている場所だけ**です。関数の中の `process.env[name]` は「実行してみないと何の名前か分からない」ので置き換えられず、ブラウザには `process.env` の中身が無いため `undefined` になります。

OK: 使う変数ごとに `process.env.NEXT_PUBLIC_～` と書き、1か所にまとめておく

コピー

```
// lib/publicEnv.ts（OK: 名前をそのまま書いておけば、ビルド時に値へ置き換わる）
export const publicEnv = {
  // アプリ名（実験用）
  appName: process.env.NEXT_PUBLIC_APP_NAME,
  // Supabase の URL
  supabaseUrl: process.env.NEXT_PUBLIC_SUPABASE_URL,
};
```

> **環境変数を変更したら必ず「再デプロイ」が必要な理由:** Next.js はビルド時に環境変数の値をコードの中に焼き付けます。そのため、Vercel ダッシュボードで環境変数の値を変えただけでは反映されません。**変更後は「Redeploy」ボタンを押して、新しい値で再ビルドする必要がある**ということを覚えておきましょう。これは初心者が「変えたのに反映されない！」とハマる典型ポイントです。

実験2: 値を変えても、ビルドし直すまでブラウザ用の JS は古いまま

> **▼ このコードがやること（先に日本語で）：** `.env.local` の Supabase の URL を書き換えて、**ビルドし直す前と後**で、ブラウザ用の JS に新しい値が入っているかを検索して比べます。「値を変えたのに反映されない」の正体を、目で確かめる実験です。

コピー

```
# 1) .env.local の NEXT_PUBLIC_SUPABASE_URL を新しい値に書き換えて保存しておく
#    （例: https://xyzbookdemo123.supabase.co → https://newbookdemo456.supabase.co）
# 2) ビルドし直す「前」に、新しい値を探す
Select-String -Path .next\static\chunks\*.js -Pattern "newbookdemo456" -List | Select-Object Filename
# 3) ビルドし直す
npm run build
# 4) ビルドし直した「後」に、もう一度探す
Select-String -Path .next\static\chunks\*.js -Pattern "newbookdemo456" -List | Select-Object Filename
```

▼ 実行結果（ビルドの出力は省略）

コピー

```
PS> Select-String -Path .next\static\chunks\*.js -Pattern "newbookdemo456" -List | Select-Object Filename
PS> npm run build
PS> Select-String -Path .next\static\chunks\*.js -Pattern "newbookdemo456" -List | Select-Object Filename

Filename
--------
2z5twj0x3ydak.js
```

- ビルドし直す**前**は、新しい値はどこにもありません。ブラウザ用の JS には、前回ビルドしたときの古い URL が書き込まれたままです。
- ビルドし直した**後**に、やっと新しい値が入りました。Vercel の「Redeploy」は、この「ビルドし直し」を Vercel のサーバー上で行うボタンです。

> **補足:** Vercel には、もう1つ「デプロイごとに環境変数の値を固定する」仕組みがあります。公式ドキュメントにも「環境変数の変更は、それ以前のデプロイには反映されず、新しいデプロイにだけ反映される」と書かれています。サーバーだけで使う変数でも同じなので、「**Vercel で環境変数を変えたら Redeploy**」と覚えておけば間違いありません。

0.4 開発・本番の3つの違い

項目	開発（npm run dev）	本番（npm run build → vercel）
URL	`http://localhost:3000`	`https://your-app.vercel.app`
速度	遅い（毎回コンパイル）	速い（最適化済み）
エラー表示	詳細なスタックトレース	ユーザーには非表示
ホットリロード	あり（保存で即反映）	なし
ファイルサイズ	大きい	小さい（圧縮済み）
環境変数の出元	`.env.local`	Vercel の設定画面

ローカルと本番でハマるポイント:

- **環境変数の設定漏れ**: `.env.local` には書いたが、Vercel 側に登録し忘れる。本番で「supabaseUrl is required」エラーになる典型例
- **大文字小文字の違い**: Windows/Mac はファイル名の大文字小文字を区別しないが、Vercel（Linux）は厳密に区別する。`BookCard.tsx` と `bookcard.tsx` は別ファイル扱い
- **`window` や `localStorage` をサーバー側で参照**: 開発時には気づかなくても、本番のServer Componentでは `window is not defined` エラーになる
- **タイムゾーン**: ローカル（日本時間）と Vercel のサーバー（UTC）で日時の扱いが違う場合がある
- **ビルド時にしか実行されないコード**: `generateStaticParams` などはローカルでは毎回実行されるが、本番では1回しか実行されないので注意

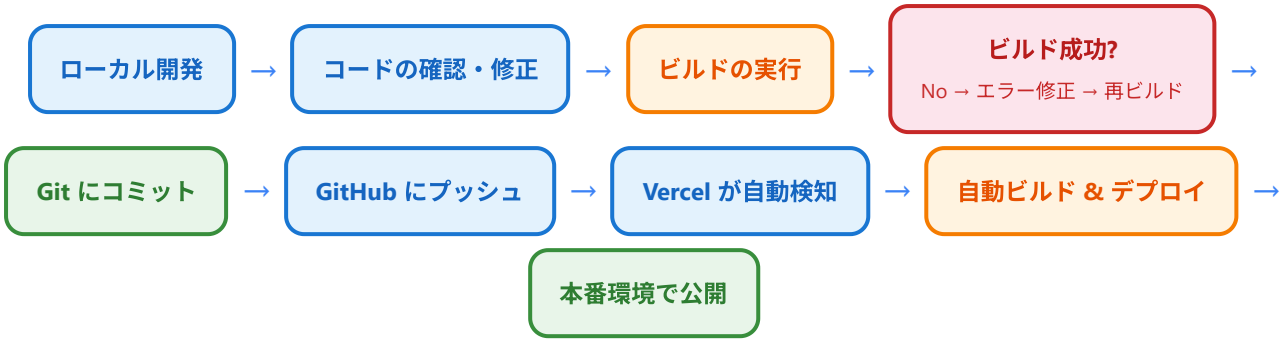
---

1. デプロイの準備

アプリケーションを本番環境にデプロイする前に、いくつかの準備が必要です。ローカル環境では動いていても、本番環境では動かないケースがあるため、事前にしっかり確認しましょう。

1.1 デプロイまでの全体フロー

まず、開発からデプロイまでの全体的な流れを確認しましょう。



1.2 ビルドの実行

Next.js アプリケーションをデプロイする前に、必ずローカルでビルドを実行して問題がないか確認します。

コピー

```
# プロジェクトのルートディレクトリ（package.json があるフォルダ）で実行する
# npm = Node Package Manager（Node.js のパッケージ管理ツール）
# run = scripts に書かれたコマンドを実行する
# build = scripts の "build" を実行する（実体は "next build"）
npm run build
```

ビルドが成功すると、以下のような出力が表示されます。これは、この教材の書籍管理アプリと同じページ構成のプロジェクトを、Next.js 16.4.1 で実際にビルドしたときの出力です。

コピー

```
> book-management@0.1.0 build
> next build

▲ Next.js 16.4.1 (Turbopack)
- Environments: .env.local
✓ Running next.config.ts took 27ms

  Creating an optimized production build ...
✓ Compiled successfully in 3.9s
  Running TypeScript ...
  Finished TypeScript in 1666ms ...
  Collecting page data using 8 workers ...
  Generating static pages using 8 workers (0/5) ...
  Generating static pages using 8 workers (1/5)
  Generating static pages using 8 workers (2/5)
  Generating static pages using 8 workers (3/5)
✓ Generating static pages using 8 workers (5/5) in 630ms
  Finalizing page optimization ...

Route (app)
┌ ƒ /
├ ○ /_not-found
├ ƒ /books/[id]
├ ƒ /books/[id]/edit
└ ○ /books/new


○  (Static)   prerendered as static content
ƒ  (Dynamic)  server-rendered on demand
```

ビルドとは？ 開発中のコード（TypeScript、JSX など）をブラウザが直接理解できる形式（JavaScript、HTML、CSS）に変換し、最適化する処理のことです。ビルドによってファイルサイズが小さくなり、読み込み速度が向上します。

▼ 出力の読み方

出力の行	意味
`▲ Next.js 16.4.1 (Turbopack)`	使っている Next.js の版と、変換に使う道具（Turbopack：ターボパック）
`- Environments: .env.local`	`.env.local` を読み込んだという印。この行が無ければ、環境変数を1つも読めていない
`✓ Compiled successfully`	TypeScript や JSX を、JavaScript に変換できた
`Running TypeScript ...`	型チェック。型エラーがあると、ここで止まる（1.3 のエラー1）
`Generating static pages`	先に作っておけるページの HTML を作っている
`Route (app)` の表	ページの一覧。`○` は静的、`ƒ` は動的

- `/`（書籍一覧）が `ƒ` なのは、第7章で作ったサーバー用の Supabase クライアント（`lib/supabase/server.ts`）が `cookies()` を使うからです。Cookie はアクセスしてきた人ごとに違うので、Next.js は「このページはアクセスのたびにサーバーで作る」と判断します。だからこそ、本を追加すると一覧にすぐ反映されます。
- `/books/new`（新規登録）が `○` なのは、`"use client"` のフォーム画面で、最初に表示する HTML が誰に対しても同じだからです。データベースへの書き込みは、ボタンを押したあとにブラウザから行われます。
- `/books/[id]` と `/books/[id]/edit` は、URL の `[id]` ごとに中身が変わるので `ƒ` です。
- `/_not-found` は、存在しない URL を開いたときの「404 ページ」で、Next.js が自動で用意してくれます。

> **用語:** 静的（Static）… ビルドのときに HTML を作り置きしておく方式。動的（Dynamic）… アクセスがあるたびにサーバーで HTML を作る方式。

1.2.1 本番モードを手元で動かす（npm run start）

`npm run build` が通っても、まだ安心はできません。ビルドしたもの（`.next/` フォルダ）を実際に動かし、ブラウザで全ページを触ってみるのが確実です。`npm run start` を使うと、Vercel にデプロイしたときとほぼ同じ「本番モード」を自分のPCで再現できます。

> **▼ このコードがやること（先に日本語で）**：ビルド結果を使って、本番モードの Web サーバーを手元で起動します。`npm run dev` と違ってファイルを保存しても画面は自動で変わりませんが、その代わり本番と同じ速さ・同じ動きで確認できます。

コピー

```
# 1) 本番用にビルドする（まだなら）
npm run build
# 2) ビルド結果を使って、本番モードでサーバーを起動する（ポート番号 3001 を指定）
npm run start -- -p 3001
```

▼ **実行結果**（`Network` の行の IP アドレスは一部を伏せています）

コピー

```
> book-management@0.1.0 start
> next start -p 3001

▲ Next.js 16.4.1
- Local:         http://localhost:3001
- Network:       http://192.168.x.x:3001
✓ Ready in 175ms
✓ Running next.config.ts took 39ms
```

▼ コードを1つずつ分解して解説

解説1: `npm run start` **の中身**

コピー

```
npm run start -- -p 3001
```

- `npm run start` は、`package.json` の `"start": "next start"` を実行します。`next start` は「ビルド済みの `.next/` を使って本番モードのサーバーを起動する」コマンドです。ビルドしていない状態で実行するとエラーになるので、必ず `npm run build` の後に使います。
- `--` は「ここから後ろは npm ではなく `next start` に渡すオプション」という区切りの記号です。`-p 3001` で、使うポート番号を 3001 に変えています。何も付けなければ 3000 番です。
- `✓ Ready` と表示されたら、ブラウザで `http://localhost:3001` を開き、一覧・登録・詳細・編集・削除をひととおり試します。止めるときは、そのターミナルで `Ctrl + C` を押します。

> **用語:** ポート … 1台のPCの中で「どのプログラムに通信を届けるか」を区別する番号。1つの番号は、同時に1つのプログラムしか使えません。

ここで、本番モードを初めて動かすときによくある失敗を1つ見ておきましょう。

> **つまずきポイント:** `npm run dev` を別のターミナルで動かしたまま `npm run start` を実行すると、3000 番がすでに使われているため起動に失敗します。

NG: 開発サーバーを止めずに、同じ 3000 番で本番モードを起動する

コピー

```
npm run start
```

コピー

```
> book-management@0.1.0 start
> next start

× Failed to start server
Error: listen EADDRINUSE: address already in use :::3000
    at new Promise (<anonymous>) {
  code: 'EADDRINUSE',
  errno: -4091,
  syscall: 'listen',
  address: '::',
  port: 3000
}
```

`EADDRINUSE`（address already in use）は「その住所（ポート）はもう使われています」という意味です。

OK: 開発サーバーを `Ctrl + C` で止めてから `npm run start` するか、別の番号を指定する

コピー

```
npm run start -- -p 3001
```

1.2.2 つまずき: Cache Components を有効にしているとビルドが失敗する

第6章では、`create-next-app` の質問で **Cache Components は No** を選びました。もし Yes を選んだ（または推奨設定のまま作った）場合、`next.config.ts` に `cacheComponents: true` が入り、この教材のトップページ（`/`）が `npm run build` で失敗します。しかも `npm run dev` では画面が普通に表示されるので、**デプロイの直前まで気づかない**のがやっかいな点です。

NG: `cacheComponents: true` が入った `next.config.ts`

コピー

```
// next.config.ts（NG: Cache Components が有効）
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
  // 新しいキャッシュの仕組み（Cache Components）を有効にする設定
  cacheComponents: true,
  // Cache Components と組み合わせて使う先読みの設定
  partialPrefetching: true,
  turbopack: {
    rules: {
      "*.css": {
        loaders: ["@tailwindcss/turbopack"],
        as: "*.css",
      },
    },
  },
};

export default nextConfig;
```

▼ 実行結果（`npm run build`。抜粋）

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
▲ Next.js 16.4.1 (Turbopack)
- Environments: .env.local
✓ Running next.config.ts took 38ms
- Cache Components enabled
- Partial Prefetching enabled

  Creating an optimized production build ...
✓ Compiled successfully in 3.7s
  Running TypeScript ...
  Finished TypeScript in 3.2s ...
  Collecting page data using 9 workers ...
  Generating static pages using 9 workers (0/8) ...
  Generating static pages using 9 workers (2/8)
Error: Route "/": Next.js encountered uncached or runtime data during prerendering.

`fetch(...)`, `cookies()`, `headers()`, `params`, `searchParams`, or `connection()` accessed outside of
`<Suspense>` prevents the route from being prerendered, blocking the page load and leading to a slower user
experience.

Ways to fix this:
  - [stream] Provide a placeholder with `<Suspense fallback={...}>` around the data access
  - [cache] For uncached data (`fetch`, database calls): cache the access with `"use cache"` (does not
apply to `connection()`)
  - [block] Set `export const instant = false` to allow a blocking route

Learn more: https://nextjs.org/docs/messages/blocking-prerender-dynamic
    at body (<anonymous>)
    at html (<anonymous>)
（中略）
Error occurred prerendering page "/". Read more: https://nextjs.org/docs/messages/prerender-error
Export encountered an error on /page: /, exiting the build.
× Next.js build worker exited with code: 1 and signal: null
```

- エラー文は「`/` のページで、`cookies()` などのデータを `<Suspense>` の外で使っているので、ページを事前に作れない」という意味です。Cache Components は「どこまでを作り置きし、どこからを毎回作るか」を細かく書き分ける新しい仕組みで、この教材のコードはその書き方に合わせていません。
- `npm run dev` で同じページを開くと、画面は表示されますが、ターミナルに同じ種類のエラー（`Next.js encountered uncached data during prerendering.`）が出ています。開発中もターミナルの赤い文字を見逃さないようにしましょう。

OK: `cacheComponents` と `partialPrefetching` の2行を消す（create-next-app で Cache Components に No と答えたときと同じ内容になります）

コピー

```
// next.config.ts（OK: Cache Components を使わない）
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
  turbopack: {
    rules: {
      "*.css": {
        loaders: ["@tailwindcss/turbopack"],
        as: "*.css",
      },
    },
  },
};

export default nextConfig;
```

保存して `npm run build` をやり直すと、1.2 の出力のように最後まで成功します。プロジェクトを作り直せる段階なら、第6章の手順で Cache Components に **No** と答えて作り直しても構いません。

1.3 よくあるビルドエラーと解決方法

ビルド時にエラーが出ることはよくあります。エラーが出たからといって自分のスキルを疑う必要はなく、エラーは「**どこを直せばよいか教えてくれるヒント**」だと考えましょう。以下は、初心者がよく遭遇するエラーとその解決方法です。

ビルドエラーメッセージの読み方

エラーメッセージは英語で出ますが、構造はだいたい同じです。次の3点を順番に確認すれば、ほとんどのエラーは特定できます。

1. **エラーの種類**: `Type error`、`Module not found`、`SyntaxError` など、一番上の行
2. **ファイル名と行番号**: `./components/BookCard.tsx:12:5` のように出る部分（12行目の5文字目）
3. **エラーの詳細メッセージ**: 何が問題なのか具体的に書かれている部分

本物の出力で練習してみましょう。次の図は、Next.js 16.4.1 で `BookCard.tsx` の `book.title` を、わざと `book.titel` と打ち間違えてビルドしたときの出力です。



図10-6: ビルドエラーの読み方（実際の出力を ①ファイル ②行と列 ③種類 ④内容 に分解）

> **ヒント**: Next.js 15 では `./components/BookCard.tsx:7:39` と `Type error: ...` の形、16 では `components/BookCard.tsx(7,39): error TS2551: ...` の形で表示されます。見た目は違っても「ファイル名・行・列・種類・内容」が入っている点は同じです。

エラー1: TypeScript の型エラー

コピー

```
Type error: Property 'title' does not exist on type 'Book'.
```

原因: 型定義と実際のコードが一致していない。

解決方法:

コピー

```
// 型定義を確認して修正する
// type は「型エイリアス」と呼ばれ、Book という名前で型を定義する構文
type Book = {
  // 書籍ID。文字列型（UUID形式の文字列を想定）
  id: string;
  // ← このプロパティが定義されているか確認。書籍タイトル
  title: string;
  // 著者名。文字列型
  author: string;
  // 評価。数値型（1〜5を想定）
  rating: number;
  // 作成日時。Supabase が ISO 8601 形式の文字列で返す
  created_at: string;
};
```

▼ **実際の出力**（Next.js 16.4.1。`BookCard.tsx` で `book.title` を `book.titel` と打ち間違えた場合）

コピー

```
  Running TypeScript ...
components/BookCard.tsx(7,39): error TS2551: Property 'titel' does not exist on type 'Book'. Did you mean 'title'?
Failed to type check.
```

> **つまずきポイント:** 型エラーの多くは「型定義が間違っている」のではなく、**使う側の打ち間違い**です。`Did you mean 'title'?`（もしかして title のこと？）というヒントが出ていたら、まずはその通りに直してみましょう。型定義の方を書き換えて黙らせるのは最後の手段です。

NG: プロパティ名を打ち間違えている

コピー

```
// components/BookCard.tsx の一部（NG: titel は Book 型に無い）
<h2 className="font-bold">{book.titel}</h2>
```

OK: 型定義どおりの名前を使う

コピー

```
// components/BookCard.tsx の一部（OK: Book 型の title）
<h2 className="font-bold">{book.title}</h2>
```

エラー2: import エラー

コピー

```
Module not found: Can't resolve '@/components/BookCard'
```

原因: ファイルパスが間違っている、またはファイルが存在しない。

解決方法:

コピー

```
# ls = list（一覧表示）コマンド。指定したファイルが存在するか確認するために使う
# 存在しない場合は「No such file or directory」と表示される
ls components/BookCard.tsx

# ファイル名の大文字・小文字も確認（Linux ではケースセンシティブ）
# BookCard.tsx と bookCard.tsx は別のファイルとして扱われます
# Windows/Mac の開発環境では区別されないため、ローカルで動いても Vercel（Linux）で失敗する典型例
```

▼ 実際の出力（Next.js 16.4.1。`app/page.tsx` で `@/components/BookCard` を `@/components/bookCard` と小文字で書いた場合）

コピー

```
> Build error occurred
Error: Turbopack build failed with 1 error:
./app/page.tsx:2:1
Error: Module not found: Can't resolve '@/components/bookCard'
  1 | import { createClient } from "@/lib/supabase/server";
> 2 | import BookCard from "@/components/bookCard";
    | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  3 | import type { Book } from "@/types/book";
  4 |
  5 | export default async function HomePage() {

Import map: aliased to relative './components/bookCard' inside of [project]/


https://nextjs.org/docs/messages/module-not-found
```

Next.js 16 の Turbopack は Windows でも大文字・小文字を区別して探すので、この打ち間違いは手元のビルドで見つかります。やっかいなのは、次の「Git が気づかない」ケースです。

> **つまずきポイント**: ファイル名の**大文字・小文字だけ**を変えたとき（例: `bookCard.tsx` → `BookCard.tsx` ）、Windows や Mac の Git は、その変更に気づきません。手元では新しい名前で動くのに、GitHub には古い名前のまま残るため、**Vercel のビルドだけが `Module not found` で失敗します。**

NG: エクスプローラーやエディタで名前だけ変えて、そのままコミットする

コピー

```
# 名前を変えたあとで、Git が変更に気づいているか確かめる
git status --short
# Git が覚えているファイル名を表示する
git ls-files components
```

コピー

```
PS> git status --short
PS> git ls-files components
components/bookCard.tsx
```

`git status` には何も出ず（＝変更なしと思われている）、Git はまだ古い `bookCard.tsx` という名前を覚えています。

OK: 名前の変更は `git mv` で Git に伝える（すでに名前を変えたあとでも、このコマンドで伝えられます）

コピー

```
# 古い名前 → 新しい名前 の順に書く
git mv components/bookCard.tsx components/BookCard.tsx
git status --short
```

コピー

```
PS> git mv components/bookCard.tsx components/BookCard.tsx
PS> git status --short
R  components/bookCard.tsx -> components/BookCard.tsx
```

先頭の `R` は Renamed（名前の変更）の意味です。これをコミットして push すれば、GitHub と Vercel にも新しい名前が届きます。

エラー3: 環境変数が undefined

コピー

```
Error: supabaseUrl is required.
```

原因: 環境変数が設定されていない、または `.env.local` が読み込まれていない。

解決方法:

コピー

```
# cat = ファイルの中身を表示するコマンド（concatenate の略）
# .env.local の中身を一覧で確認できる
cat .env.local

# 以下の変数が設定されているか確認
# NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
# NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_...（旧 anon key なら eyJhbGciOi... で始まる長い文字列）
# 注意：環境変数を書き換えた後は、開発サーバー（npm run dev）を停止して再起動が必要
```

> **補足（この教材のアプリの場合）**：第7章で作った `lib/supabase/server.ts`（`@supabase/ssr` を使う版）では、環境変数が無いときのエラー文は `Your project's URL and Key are required to create a Supabase client!` になります。しかもこのエラーは**ビルドのときではなく、ページを開いたとき（実行時）**に出ます。ビルドが成功しても安心できない理由と、ビルドの時点で気づくための工夫は「1.4.1」と「2.9.2」で扱います。

エラー4: 'use client' ディレクティブの不足

コピー

```
Error: useState only works in Client Components. Add the "use client" directive.
```

原因: Client Component のフック（`useState`, `useEffect` など）を Server Component で使っている。

解決方法:

コピー

```
// ファイルの先頭（必ず1行目）に追加
// この一行があると、Next.js はこのファイルを Client Component として扱う
// Server Component（デフォルト）はサーバー側でしか動かないので、useState や useEffect が使えない
'use client';

// React の useState フックを使うために import する
import { useState } from 'react';
// ...
```

▼ 実際の出力（Next.js 16.4.1。`app/books/new/page.tsx` の1行目の `"use client";` を消した場合。抜粋）

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは →

```
> Build error occurred
Error: Turbopack build failed with 2 errors:
./app/books/new/page.tsx:2:10
Error: You're importing a module that depends on `useRouter` into a React Server Component module. This API is only available in Client Components. To fix, mark the file (or its parent) with the `"use client"` directive.
    Learn more: https://nextjs.org/docs/app/api-reference/directives/use-client
  1 | import { useState } from "react";
> 2 | import { useRouter } from "next/navigation";
    |          ^^^^^^^^^
  3 | import { createClient } from "@/lib/supabase/client";
  4 |
  5 | export default function NewBookPage() {

Ecmascript file had an error
```

同じ種類のエラーが、`useState` についてももう1つ出ます。エラー文の最後に `To fix, mark the file (or its parent) with the "use client" directive.`（直すには、ファイルに "use client" を付けてください）と、直し方そのものが書かれています。

エラー5: ESLint エラー

```
ESLint: 'variable' is defined but never used. (@typescript-eslint/no-unused-vars)
```

原因: 使用されていない変数やインポートがある。

解決方法:

```
// 不要な変数・インポートを削除する
// または、意図的に未使用の場合はアンダースコアを付けると ESLint が無視してくれる
// アンダースコア（_）始まりの変数は「使わないことを明示している」という慣習
const _unusedVariable = 'something';
```

> **注意（Next.js 16 で変わった点）**: Next.js 15 までは、`npm run build` の途中で ESLint も自動で実行されていました。**Next.js 16 からは、ビルドで ESLint が実行されなくなりました**（`next lint` コマンドも廃止され、`package.json` の `"lint"` は `"eslint"` になっています）。つまり、ESLint の問題があってもビルドは成功してしまいます。デプロイ前には `npm run lint` を**別に**実行する習慣をつけましょう。

```
# ESLint だけを実行する（ビルドとは別のコマンド）
npm run lint
```

▼ 実行結果（使っていない変数がある場合）（ファイルの場所は一部を伏せています）

コピー

```
> book-management@0.1.0 lint
> eslint


C:\...\book-management\components\BookCard.tsx
  4:7  warning  'unusedMessage' is assigned a value but never used  @typescript-eslint/no-unused-vars

✖ 1 problem (0 errors, 1 warning)
```

`warning`（警告）だけなら、コマンドは成功扱い（終了コード 0）で終わり、ビルドも止まりません。ただし「使っていない変数」は消し忘れのサインなので、見つけたら直しておきましょう。一方、次のような `error` があると、`npm run lint` は失敗扱い（終了コード 1）になります。

> **つまずきポイント:** React のフック（`useState` など）は、`if` や `return` より**前**で、毎回同じ順番で呼ぶ決まりです。途中で `return` するコードの後ろにフックを書くと、ESLint が `error` を出します。

NG: 早めの `return` のあとで `useState` を呼んでいる

コピー

```
"use client";

import { useState } from "react";

export default function Stars({ editable }: { editable: boolean }) {
  if (!editable) {
    return <span>★★★</span>;
  }
  const [rating, setRating] = useState(3);
  return <button onClick={() => setRating(rating + 1)}>{rating}</button>;
}
```

▼ 実行結果（`npm run lint`）

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは →　コピー

```
C:\...\book-management\components\Stars.tsx
  9:31  error  React Hook "useState" is called conditionally. React Hooks must be called in the exact same
order in every component render. Did you accidentally call a React Hook after an early return?  react-
hooks/rules-of-hooks

✖ 1 problem (1 error, 0 warnings)
```

OK: フックを関数のいちばん上に移す

コピー

```
"use client";

import { useState } from "react";

export default function Stars({ editable }: { editable: boolean }) {
  // OK: フックは if や return より前、関数のいちばん上で呼ぶ
  const [rating, setRating] = useState(3);
  if (!editable) {
    return <span>★★★</span>;
  }
  return <button onClick={() => setRating(rating + 1)}>{rating}</button>;
}
```

直したあとで `npm run lint` を実行すると、`> eslint` の行のあとに何も表示されずに終わります（問題 0 件）。

エラー6（エラーにならない落とし穴）：`params` の古い書き方

Next.js 15 から、ページに渡される `params`（URL の `[id]` の部分）は **Promise**（あとから値が届く箱）になりました。インターネット上の古い記事のように `params.id` と直接読むと、Next.js 16 では**ビルドも型チェックも通ってしまい**、画面で `id` が空っぽになります。エラーが出ない分、気づきにくい落とし穴です。

> **つまずきポイント:**「ビルドが通った＝正しく動く」ではありません。動的なページ（`[id]` を含むページ）は、ビルドのあとに必ず実際の URL で開いて確かめましょう。

NG: Next.js 14 までの書き方

コピー

```
// app/books/[id]/edit/page.tsx（NG: params を Promise として扱っていない）
type Props = {
  params: { id: string };
};

export default function EditBookPage({ params }: Props) {
  return (
    <main className="p-6">
      <h1 className="text-xl font-bold">書籍 {params.id} を編集</h1>
    </main>
  );
}
```

▼ 実行結果（`npm run build` は成功する。本番モードで `/books/42/edit` を開いたときの見出し）

コピー

```
書籍  を編集
```

`42` が入るはずの場所が空っぽです。`params` は Promise なので、`params.id` は `undefined` になっていました。

OK: `Promise<...>` 型にして、`await params` で中身を取り出す

コピー

```
// app/books/[id]/edit/page.tsx（OK: Next.js 15 以降の書き方）
type Props = {
  // params は Promise（あとから値が届く箱）
  params: Promise<{ id: string }>;
};

export default async function EditBookPage({ params }: Props) {
  // await で箱の中身を取り出してから id を使う
  const { id } = await params;
  return (
    <main className="p-6">
      <h1 className="text-xl font-bold">書籍 {id} を編集</h1>
    </main>
  );
}
```

その他、本番でハマる典型エラー:

- `Hydration failed because the initial UI does not match...` : サーバー側で生成したHTMLとクライアント側のレンダリング結果が違うときに出る。日時表示やランダム値が原因のことが多い
- `Cannot find module 'xxx'` : パッケージのインストール忘れ。`npm install` を再実行する
- `EACCES: permission denied` : ファイルのアクセス権限の問題。再起動や `node_modules` の削除で直ることが多い

1.4 環境変数の確認

デプロイ前に、必要な環境変数がすべて揃っているか確認しましょう。

コピー

```
# .env.local の内容を確認する
# cat コマンド：ファイルの内容を画面に表示する
cat .env.local
```

必要な環境変数一覧:

変数名	説明	例
`NEXT_PUBLIC_SUPABASE_URL`	Supabase プロジェクトの URL	`https://abcdefg.supabase.co`
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase の公開用キー（Publishable key）	`sb_publishable_...`（旧 anon key なら `eyJhbGciOi...`）

> **重要**: `.env.local` は `.gitignore` に含まれているため、GitHub にはプッシュされません。これはセキュリティ上正しい動作です。**もし `.env.local` を間違って GitHub にプッシュしてしまうと、世界中の人にキーが見られてしまう**ため、`.gitignore` で守られている、ということを覚えておきましょう。デプロイ先（Vercel）で別途環境変数を設定する必要があります。

1.4.1 環境変数のチェックを自動にする（prebuild スクリプト）

「1.3 エラー3」の補足で見たとおり、この教材のアプリは環境変数が無くても**ビルドは成功してしまい**、本番でページを開いた瞬間に初めてエラーになります。そこで、ビルドの直前に「必要な環境変数がそろっているか」を自動で確かめる小さなスクリプトを用意しましょう。お店の開店前に、レジや釣り銭がそろっているかを確認する「開店前チェック」のようなものです。

> **▼ このコードがやること（先に日本語で）**：必要な環境変数の名前を一覧にしておき、1つでも値が入っていなければ、分かりやすいメッセージを出して止まります。URL の最後に `/` を付けてしまう、などのよくある書き間違いも見つけます。`package.json` に登録すると、`npm run build` のたびに（Vercel 上のビルドでも）自動で実行されます。

コピー

```
// scripts/check-env.mjs
// ビルドの前に「必要な環境変数がそろっているか」を確かめるスクリプト

// アプリが動くために必要な環境変数の名前
const required = [
  "NEXT_PUBLIC_SUPABASE_URL",
  "NEXT_PUBLIC_SUPABASE_ANON_KEY",
];

// 手元では .env.local を読み込む（Vercel 上にはこのファイルが無い）
try {
  process.loadEnvFile(".env.local");
} catch {
  // ファイルが無ければ何もしない（Vercel では値が最初から入っている）
}

// 値が入っていない名前だけを集める
const missing = required.filter((name) => !process.env[name]);

if (missing.length > 0) {
  console.error("✕ 次の環境変数が設定されていません:");
  for (const name of missing) {
    console.error(`  - ${name}`);
  }
  console.error("→ 手元なら .env.local、Vercel なら Environment Variables を確認してください");
  // 終了コード 1 は「失敗」の合図。npm はここで止まり、ビルドに進まない
  process.exit(1);
}

// URL のよくある書き間違い（末尾の / や http:）もついでに確かめる
const url = process.env.NEXT_PUBLIC_SUPABASE_URL;
if (!url.startsWith("https://") || url.endsWith("/")) {
  console.error(`✕ NEXT_PUBLIC_SUPABASE_URL の形が正しくありません: ${url}`);
  console.error("→ https://xxxx.supabase.co の形（最後に / を付けない）にしてください");
  process.exit(1);
}

console.log(`✓ 必要な環境変数 ${required.length} 個はすべて設定されています`);
```

`package.json` の `"scripts"` に、`"prebuild"` の1行を足します。

コピー

```
{
  "scripts": {
    "dev": "next dev",
    "prebuild": "node scripts/check-env.mjs",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  }
}
```

▼ **実行結果1**（`.env.local` が無い状態で `npm run build`）

コピー

```
> book-management@0.1.0 prebuild
> node scripts/check-env.mjs

✕ 次の環境変数が設定されていません:
  - NEXT_PUBLIC_SUPABASE_URL
  - NEXT_PUBLIC_SUPABASE_ANON_KEY
→ 手元なら .env.local、Vercel なら Environment Variables を確認してください
```

▼ **実行結果2**（URL の最後に `/` が付いている場合）

コピー

```
> book-management@0.1.0 prebuild
> node scripts/check-env.mjs

✕ NEXT_PUBLIC_SUPABASE_URL の形が正しくありません: https://xyzbookdemo123.supabase.co/
→ https://xxxx.supabase.co の形（最後に / を付けない）にしてください
```

▼ **実行結果3**（正しく設定されている場合。このあと、いつものビルドが続く）

コピー

```
> book-management@0.1.0 prebuild
> node scripts/check-env.mjs

✓ 必要な環境変数 2 個はすべて設定されています

> book-management@0.1.0 build
> next build

▲ Next.js 16.4.1 (Turbopack)
- Environments: .env.local
```

▼ コードを1つずつ分解して解説

解説1: `prebuild` は build の「前」に自動で動く

コピー

```
"prebuild": "node scripts/check-env.mjs",
```

- npm には、`build` という名前のスクリプトを実行するとき、**`prebuild` という名前のスクリプトがあれば先に実行する**という決まりがあります（`pre` は「前の」という意味）。
- `prebuild` が失敗（終了コード 1）すると、`build` は実行されません。Vercel のビルドも `npm run build` で動くので、Vercel 上で環境変数を登録し忘れたときも、デプロイが **Error** で止まり、Build Logs に上のメッセージが出ます。「壊れたまま公開される」よりずっと安全です。

解説2: `.env.local` を読み込む

コピー

```
try {
  process.loadEnvFile(".env.local");
} catch {
  // ファイルが無ければ何もしない
}
```

- `npm run build` の中の Next.js は `.env.local` を自動で読みますが、このスクリプトは Next.js とは別に動く Node.js のプログラムなので、自分で読み込む必要があります。`process.loadEnvFile()` は Node.js（20.12 以降）に入っている「.env ファイルを読み込む」機能です。
- Vercel には `.env.local` が無いので読み込みに失敗しますが、`try { } catch { }` で「失敗しても気にしない」ようにしています。Vercel では、Environment Variables に登録した値が最初から `process.env` に入っています。

解説3: 足りない名前を集めて、止める

コピー

```
const missing = required.filter((name) => !process.env[name]);
```

- `filter` で、「値が入っていない（`!process.env[name]` が true になる）名前」だけを集めています。
- このスクリプトは**ブラウザではなく Node.js で動く**ので、`process.env[name]` のように名前を変数で渡しても正しく読めます（0.3 のつまずきポイントは、ブラウザ用の JS に限った話です）。
- `process.exit(1)` は「失敗としてプログラムを終える」命令です。

1.4.2 `.env.local` が Git に入っていないか確かめる

`.env.local` が `.gitignore` で守られているかは、思い込みではなく**コマンドで確かめる**のが確実です。

> **▼ このコードがやること（先に日本語で）:**「`.env.local` が Git に無視されているか」「どの行のルールで無視されているか」「Git の管理対象に入ってしまっていないか」を、3つのコマンドで確認します。

コピー

```
# どのルールで無視されているかを表示する（-v は詳しく表示するオプション）
git check-ignore -v .env.local
# 変更されたファイルの一覧（ここに .env.local が出てはいけない）
git status --short
# Git が管理しているファイルの中に .env.local があるか（何も出なければ OK）
git ls-files .env.local
```

▼ 実行結果（正しく守られている場合）

コピー

```
PS> git check-ignore -v .env.local
.gitignore:34:.env* .env.local
PS> git status --short
PS> git ls-files .env.local
PS>
```

- `.gitignore:34:.env*` は、「`.gitignore` の 34 行目にある `.env*` というルールで無視されている」という意味です。`*` は「何文字でもよい」という記号なので、`.env` で始まるファイルはすべて無視されます（create-next-app が作る `.gitignore` の書き方です）。
- `git status --short` と `git ls-files .env.local` に何も出なければ、`.env.local` は GitHub に送られません。

> **つまずきポイント:** `.gitignore` に書く**前**に一度でもコミットしてしまったファイルは、あとから `.gitignore` に書いても**無視されません。** Git は「すでに管理しているファイル」を追いかけ続けるからです。

NG: 先にコミットしてしまった `.env.local` の状態（実験用の小さなリポジトリで、`.env.local` をコミットしたあとに `.gitignore` を作って再現した出力です）

コピー

```
PS> git check-ignore -v .env.local
PS> git ls-files
.env.local
```

`git check-ignore` に何も出ない（＝無視されていない）うえ、`git ls-files` に `.env.local` が出ています。このまま push すると GitHub に送られてしまいます。

OK: `git rm --cached` で「管理だけ」をやめる（PCの中のファイルは消えません）

コピー

```
git rm --cached .env.local
git status --short
git check-ignore -v .env.local
```

コピー

```
PS> git rm --cached .env.local
rm '.env.local'
PS> git status --short
D  .env.local
?? .gitignore
PS> git check-ignore -v .env.local
.gitignore:1:.env*  .env.local
```

`D` は「Git の管理から外す（Deleted）」という意味です。これをコミットすれば、次からは `.env.local` は送られません。ただし、**すでに GitHub に push してしまった場合は、過去の履歴に値が残っています**。Secret key のような本当に秘密の値だったときは、Supabase の画面でそのキーを無効にして、新しいキーを作り直してください。

1.5 ビルド前チェックリスト

デプロイ前に以下をすべて確認しましょう。

- [ ] `npm run build` がエラーなく成功する
- [ ] `npm run lint` がエラーなく成功する（Next.js 16 では、ビルドとは別に実行が必要）
- [ ] `.env.local` に必要な環境変数がすべて設定されている
- [ ] `.gitignore` に `.env.local` と `node_modules` が含まれている
- [ ] ブラウザで全ページが正常に表示される
- [ ] CRUD 操作（作成・読取・更新・削除）がすべて動作する
- [ ] コンソールにエラーが出ていない
- [ ] `npm run start` の本番モードで、全ページを開いて確かめた（1.2.1）
- [ ] `next.config.ts` に `cacheComponents: true` が入っていない（1.2.2）
- [ ] `git status` や `git ls-files .env.local` に `.env.local` が出てこない（1.4.2）

---

2. Vercel へのデプロイ

2.1 Vercel とは

Vercel（ヴァーセル）は、Next.js を開発している会社が提供するホスティングプラットフォームです。Next.js アプリケーションのデプロイに最適化されており、最も簡単かつ確実にデプロイできます。

Vercel を選ぶ理由:

- Next.js の開発元が運営しているため、最新機能への対応が最速
- GitHub と連携して、プッシュするだけで自動デプロイ
- 無料プラン（Hobby）で個人プロジェクトには十分な機能
- グローバル CDN で高速配信
- SSL（HTTPS）が自動で設定される
- プレビューデプロイ（Pull Request ごとにプレビュー URL を自動生成）

2.2 ホスティングサービスの比較

サービス	Next.js 対応	無料プラン	自動デプロイ	難易度	特徴
Vercel	最適	あり	あり	簡単	Next.js 公式。最も相性が良い
Netlify	良好	あり	あり	簡単	静的サイトに強い。Next.js 対応も改善中
AWS Amplify	良好	あり（制限付き）	あり	やや難	AWS エコシステムとの統合が強み
Railway	良好	あり（制限付き）	あり	普通	バックエンドも含めた総合プラットフォーム
Cloudflare Pages	部分的	あり	あり	やや難	エッジコンピューティングに強い

> **この教材では Vercel を使用します。** Next.js との相性が最も良く、設定が最も簡単だからです。

2.3 事前準備: GitHub リポジトリの作成

Vercel にデプロイするには、コードが GitHub に存在する必要があります。まだ GitHub リポジトリを作成していない場合は、以下の手順で作成します。

Step 1: Git の初期化とコミット

コピー

```
# ------------------------------------------------------------------------
# プロジェクトのルートディレクトリ（package.json があるフォルダ）で実行する
# ------------------------------------------------------------------------

# (1) Git リポジトリを初期化（既に init 済みなら不要、何度実行してもOK）
#     カレントフォルダに隠しフォルダ .git/ が作られ、Git管理が始まる
#     git = バージョン管理ツールの名前
#     init = initialize（初期化）の略
git init
# ▼ 出力例
# Initialized empty Git repository in /path/to/book-management/.git/

# (2) ステージングエリアに全ファイルを登録
#     git add = コミット候補に追加するコマンド
#     . はカレントフォルダ全体を意味する（「すべてのファイル」と読み替えてもOK）
#     .gitignore に書かれたファイル/フォルダ（node_modules や .env.local 等）は
#     自動的に除外されるので安心。
git add .

# (3) 最初のコミット
#     commit = 「変更を記録する」操作。スナップショットを残すイメージ
#     -m "..." はコミットメッセージ。何をした履歴か後で分かるよう短文で残す。
#     メッセージなしで commit するとエディタが開く（初心者がハマりやすいポイント）
git commit -m "書籍管理アプリの初回コミット"
# ▼ 出力例
# [main (root-commit) abc1234] 書籍管理アプリの初回コミット
#  150 files changed, 12345 insertions(+)
#  create mode 100644 README.md
#  create mode 100644 package.json
#  ...
```

Step 2: GitHub にリポジトリを作成

1. GitHub にログインする
2. 右上の「+」ボタンをクリック →「New repository」を選択
3. リポジトリ名を入力（例: `book-management-app`）
4. 「Private」を選択（公開したくない場合）
5. 「Create repository」をクリック

Step 3: ローカルリポジトリと GitHub を接続

コピー

```
# ------------------------------------------------------------------------
# GitHub のリポジトリ作成画面に表示される3行のコマンド
# ------------------------------------------------------------------------

# (1) リモート接続先を「origin」という名前で登録
#     git remote add = リモート（GitHub などのサーバー側リポジトリ）を登録する命令
#     origin = リモートの「あだ名」。慣習的に最初のリモートには origin を使う
#     URL は GitHub 上のリポジトリの URL に置き換える
#     これ以降「origin」と書けばそのURLを指すようになる
git remote add origin https://github.com/あなたのユーザー名/book-management-app.git

# (2) ローカルのブランチ名を「main」に統一する（古い環境では master の場合がある）
#     git branch = ブランチ操作のコマンド
#     -M は「強制リネーム」を意味する（大文字の M）
#     現在のブランチ名を main に変える
git branch -M main

# (3) 初回プッシュ
#     git push = ローカルの履歴をリモートに送る操作
#     -u は --set-upstream の短縮形。「これ以降のデフォルトの送信先」として origin の main を覚えさせる
#     origin = リモート名（さっき登録したGitHubのURLのあだ名）
#     main = 送るブランチ名
#     これを一度やれば、次回からは git push だけで済む
git push -u origin main
# ▼ 出力例
# Enumerating objects: 150, done.
# Counting objects: 100% (150/150), done.
# ...
# To github.com:yourname/book-management-app.git
#  * [new branch]      main -> main
# branch 'main' set up to track 'origin/main'.
```

2.4 Vercel アカウントの作成

1. Vercel の公式サイト にアクセス
2. 「Sign Up」をクリック
3. 「Continue with GitHub」を選択して GitHub アカウントでログイン
4. Vercel が GitHub へのアクセス権限を要求するので「Authorize」をクリック
5. アカウント作成完了

> **注意**: GitHub アカウントでログインすることで、リポジトリとの連携が簡単になります。

2.5 デプロイ手順（Step by Step）

Step 1: 新規プロジェクトの作成

1. Vercel ダッシュボード（https://vercel.com/dashboard）にアクセス

2. 「Add New...」→「Project」をクリック（画面によっては、ダッシュボード右上の「New Project」ボタン）
3. 「Import Git Repository」セクションで、先ほど作成した `book-management-app` リポジトリを見つける
4. 「Import」をクリック

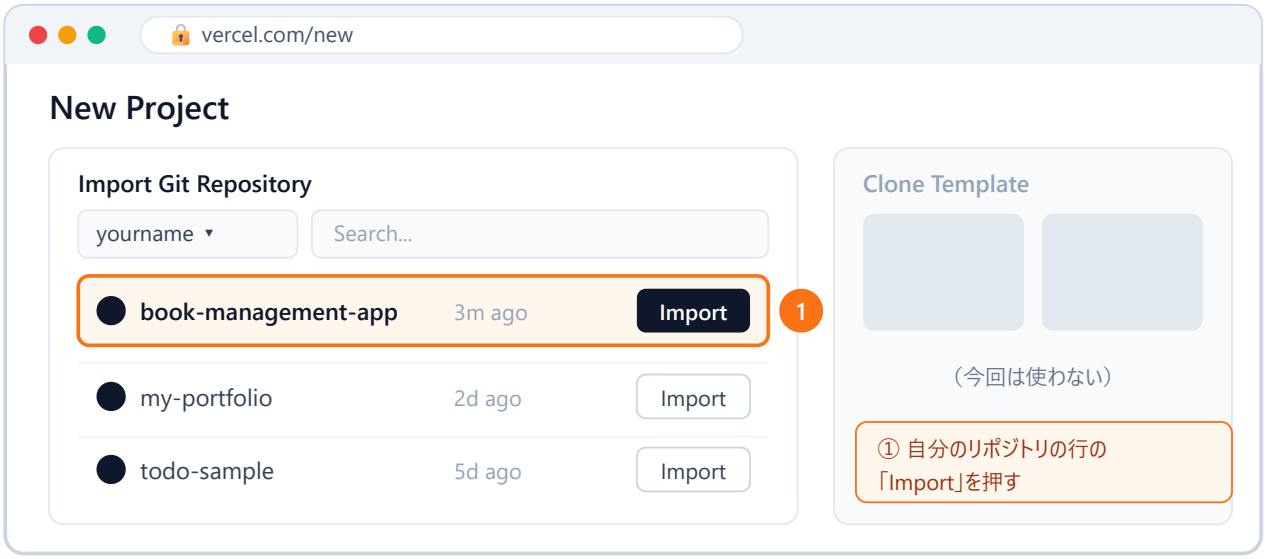


図10-7: 画面イメージ — New Project 画面でリポジトリを選ぶ（実際の見た目は Vercel の更新で変わることがあります）

> **ヒント:** 一覧に自分のリポジトリが出てこないときは、Vercel が GitHub のそのリポジトリを見る権限を持っていません。リポジトリ一覧の下などにある GitHub の設定変更のリンク（Configure GitHub App など）から、対象のリポジトリへのアクセスを許可してください。

Step 2: プロジェクトの設定

インポート画面で以下を確認します。

- **Project Name**: `book-management-app`（変更可能）
- **Framework Preset**: `Next.js`（自動検出されるはず）
- **Root Directory**: `./`（プロジェクトのルート）
- **Build Command**: `npm run build`（デフォルトのまま）
- **Output Directory**: `.next`（デフォルトのまま）

> **Build Command と Output Directory の意味:**
>
> - **Build Command**（ビルドコマンド）: Vercel のサーバー上で実行されるコマンド。デフォルトの `npm run build` は package.json の `scripts.build`（中身は `next build`）を呼び出します
> - **Output Directory**（出力ディレクトリ）: ビルド結果が保存される場所。Next.js では `.next/` フォルダに最適化されたファイルが入る。Vercel はこのフォルダの中身をサーバーに配置して公開します

画面全体は次のようになっています。Build Command と Output Directory は「Build and Output Settings」を開くと見られますが、Next.js なら触る必要はありません。この画面の下の方に、次の Step 3 で使う「Environment Variables」の欄があります。

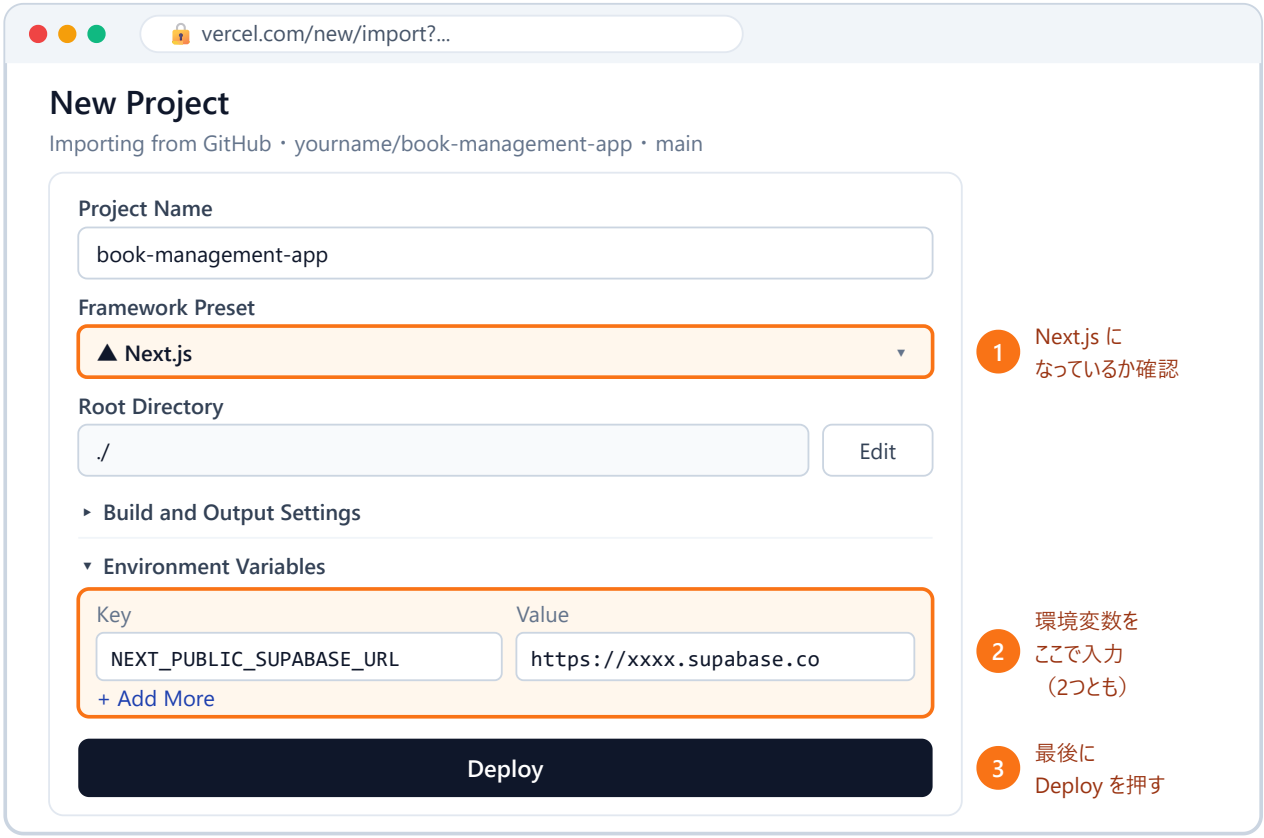


図10-8: 画面イメージ — プロジェクトの設定画面（① Framework Preset ② Environment Variables ③ Deploy）

Step 3: 環境変数の設定

これが最も重要なステップです。環境変数を設定しないと、Supabase に接続できずアプリが動作しません。

1. 「Environment Variables」セクションを展開する
2. 以下の変数を追加する

Key	Value
`NEXT_PUBLIC_SUPABASE_URL`	`https://あなたのプロジェクトID.supabase.co`
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	`sb_publishable_...（あなたの Publishable key）`

> **値の確認方法**: Supabase ダッシュボードでプロジェクトを開き、画面上部の **Connect** ボタンから Project URL と公開用キー（Publishable key）をコピーするのがいちばん簡単です。設定画面から探す場合は、**Project Settings** の **Data API**（Project URL）と **API Keys**（Publishable key）にあります（第5章）。

3. 1つ目を入力したら「Add More」で入力欄を増やし、2つとも入力する

> **つまずきポイント:** Value 欄には「値だけ」を入れます。`.env.local` の1行をまるごと貼ったり、前後に `"`（ダブルクォート）や最後に `/` を付けたりすると、Vercel はそれも含めて「値」として保存します。手元では動くのに、本番でだけ Supabase への接続に失敗する原因になります。

NG: Value に「変数名=」まで入っている／余計な記号が付いている

コピー

```
Key:   NEXT_PUBLIC_SUPABASE_URL
Value: NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co

Key:   NEXT_PUBLIC_SUPABASE_URL
Value: "https://xxxx.supabase.co/"
```

OK: Value には値だけを入れる

コピー

```
Key:   NEXT_PUBLIC_SUPABASE_URL
Value: https://xxxx.supabase.co
```

1.4.1 のチェックスクリプト（`prebuild`）を入れておけば、上の NG のような値は `https://` で始まらない、または `/` で終わるため、Vercel のビルドの時点で止まって気づけます。

> **重要：環境変数は環境ごとに設定できる:** Vercel では、環境変数を「Production（本番）」「Preview（プレビュー）」「Development（開発）」の3つの環境ごとに設定できます。多くの場合、すべての環境にチェックを入れて同じ値を入れておけばOKです。本番DBと開発DBを分けたい場合は、環境ごとに異なる値を設定することも可能です。

Step 4: デプロイの実行

1. 「Deploy」ボタンをクリック
2. Vercel がビルドとデプロイを開始する（通常 1〜3 分）
3. 成功すると、紙吹雪のアニメーションとともにデプロイ完了画面が表示される
4. 表示された URL（例: `https://book-management-app.vercel.app`）をクリックしてアプリを確認

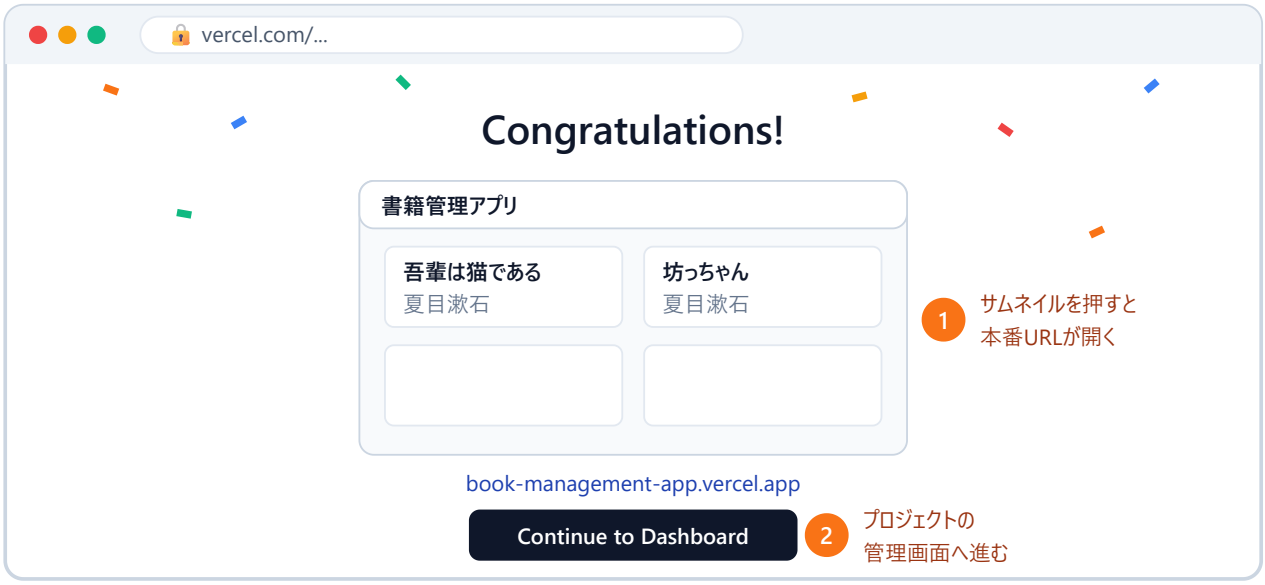


図10-9: 画面イメージ — 初回デプロイの完了画面（紙吹雪とアプリのサムネイルが表示される）

> **ヒント:** 友達に教える URL は、`book-management-app.vercel.app` のような**短い本番用の URL** にします。Deployments の一覧に出てくる `book-management-app-ランダムな英数字-yourname.vercel.app` のような長い URL は、初期設定では Vercel にログインした本人しか開けません（理由は 2.6.2 で説明します）。

（補足）Vercel CLI を使う場合

ブラウザではなくターミナルから直接デプロイしたい場合、Vercel CLI を使えます。GitHub 経由のデプロイで十分なので必須ではありませんが、参考までに紹介します。

コピー

```
# (1) Vercel CLI をグローバルインストール
#     -g は global（システム全体に入れる）オプション
#     これでターミナルのどこからでも vercel コマンドが使える
npm install -g vercel

# (2) Vercel アカウントでログイン
#     ブラウザが開き、認証フローに進む
vercel login

# (3) 現在のディレクトリをプレビュー環境にデプロイ
#     プロジェクトのルート（package.json があるフォルダ）で実行
vercel
# ↑ 初回は質問がいくつか出る（プロジェクト名・フレームワーク等）。基本Enterで進めればOK

# (4) 本番環境にデプロイ
#     --prod = Production（本番）環境にデプロイ
vercel --prod
```

（補足）vercel.json の例

プロジェクトのルートに `vercel.json` を置けば、Vercel の挙動を細かく設定できます。本書のチュートリアルでは不要ですが、参考までに。

コピー

```
{
  "framework": "nextjs",
  "buildCommand": "npm run build",
  "outputDirectory": ".next",
  "installCommand": "npm install",
  "devCommand": "npm run dev",
  "regions": ["hnd1"]
}
```

各行の意味：

- `"framework": "nextjs"` … フレームワーク種別。Vercel が最適なビルド方式を自動選択するためのヒント
- `"buildCommand": "npm run build"` … ビルド時に実行するコマンド
- `"outputDirectory": ".next"` … ビルド結果の出力先フォルダ
- `"installCommand": "npm install"` … 依存パッケージのインストール時に実行するコマンド
- `"devCommand": "npm run dev"` … 開発サーバー起動時のコマンド（`vercel dev` で使われる）
- `"regions": ["hnd1"]` … デプロイ先リージョン。`hnd1` は東京リージョン。日本のユーザーがメインなら東京を指定すると応答が速い

（補足）next.config.js の本番向け設定例

`next.config.js` は Next.js プロジェクトの設定ファイルです。本書ではほぼデフォルトでOKですが、本番向けに調整したい場合のサンプルです。

コピー

```
// JSDoc コメントで型情報を付ける（エディタの補完が効くようになる）
/** @type {import('next').NextConfig} */
const nextConfig = {
  // reactStrictMode = React の Strict Mode を有効にする
  // 開発中に副作用の問題を早期発見しやすくなる。本番では無効化されるので付けっぱなしでOK
  reactStrictMode: true,

  // poweredByHeader = レスポンスヘッダから X-Powered-By: Next.js を消す
  // 使用技術を外部に教えないことでセキュリティを少し高められる
  poweredByHeader: false,

  // compress = レスポンスをgzipで圧縮するか。デフォルトでtrue（基本そのまま）
  compress: true,

  // images = next/image コンポーネントの設定
  images: {
    // remotePatterns = 外部画像を許可するドメイン一覧
    // ここに登録されていないドメインの画像は <Image> で表示できない（セキュリティ対策）
    remotePatterns: [
      {
        // https のみ許可
        protocol: 'https',
        // Supabase の任意のサブドメインを許可
        hostname: '*.supabase.co',
        // /storage/ 以下のパスのみ許可
        pathname: '/storage/**',
      },
    ],
  },
};

// CommonJS の書き方で nextConfig を外部に公開する（Next.js が読み込む）
module.exports = nextConfig;
```

> **本番でアプリを直接起動するコマンド:** Vercel では自動で行われますが、もし自前のサーバーで動かす場合は、ビルド後に `npm start`（中身は `next start`）を実行します。これは「ビルド済みの `.next/` フォルダの中身を使って本番モードで Web サーバーを起動する」コマンドです。`npm run dev` と違い、ファイルを変更しても自動反映されません。

2.6 デプロイの自動化フロー

一度設定すれば、以降は GitHub にプッシュするだけで自動デプロイされます。



2.6.1 git push からデプロイ完了までの時系列

`git push` してから本番 URL が新しくなるまでに、Vercel の中では何が起きているのでしょうか。時間の流れに沿って並べると、次のようになります。



図10-10: git push からデプロイ完了までの時系列（時間は小さなアプリでの目安）

Vercel のダッシュボードで、プロジェクトのサイドバーから「Deployments」を開くと、デプロイごとに次のような**状態（Status）**が表示されます。

状態	意味	そのとき本番 URL に表示されるもの
Queued（キュード）	順番待ち	前のバージョン
Building（ビルディング）	ビルド中。ログが流れている	前のバージョン
Ready（レディ）	完成して公開された	**新しいバージョン**
Error（エラー）	ビルドなどに失敗した	前のバージョンのまま
Canceled（キャンセルド）	途中で中止された	前のバージョンのまま

- 大事なのは、**Ready になるまで本番は前のバージョンのまま**という点です。ビルドに失敗しても、利用者が壊れた画面を見ることはありません。逆に言うと、「push したのに変わらない」ときは、まず状態が Error になっていないかを確かめます。
- Error になったデプロイを開くと、Deployment Details の「Building」の欄に、ビルドのログが表示されます。中身は手元の `npm run build` と同じなので、1.3 の読み方がそのまま使えます。

2.6.2 Preview（プレビュー）と Production（本番）の違い

Vercel は、**どのブランチに push したか**でデプロイの行き先を変えます。`main`（本番用のブランチ）に push すると Production デプロイ、それ以外のブランチに push すると Preview デプロイになります。

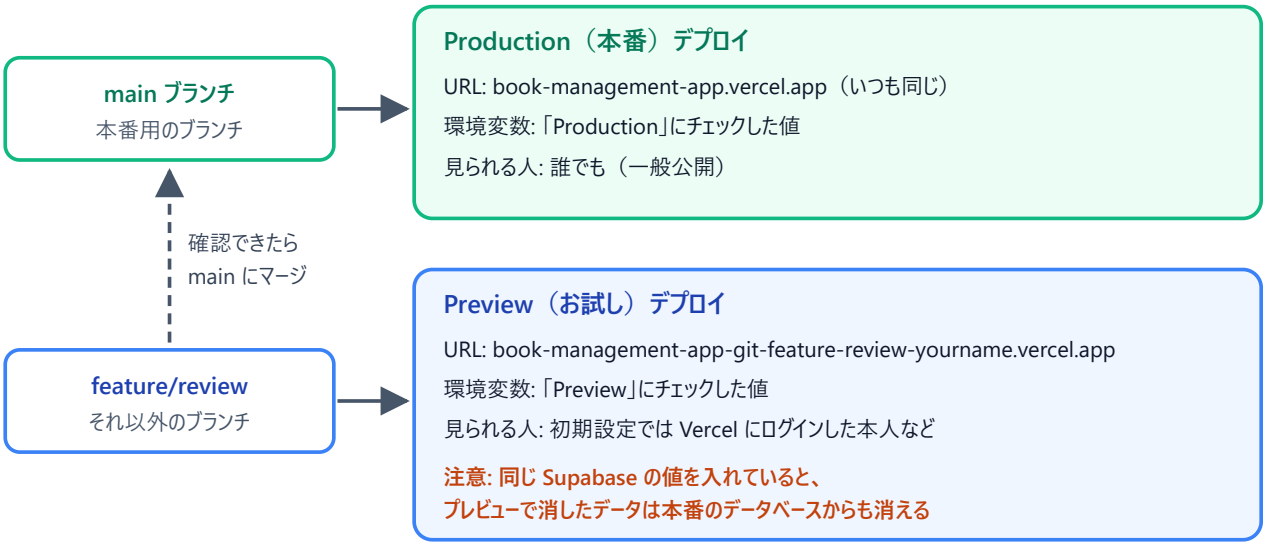


図10-11: Production と Preview の違い（どのブランチに push したかで行き先が変わる）

新しい機能を試すときは、ブランチを作って push すると、本番に影響を与えずに Preview の URL で確かめられます。

コピー

```
# 新しいブランチを作って切り替える（switch -c = 作って移動）
git switch -c feature/review
# 変更をコミットする
git add .
git commit -m "レビュー機能を追加"
# 新しいブランチを GitHub に送る（-u で次回から git push だけで送れるようにする）
git push -u origin feature/review
```

push すると Vercel が自動で Preview デプロイを作ります。GitHub で Pull Request（変更の提案）を作ると、Vercel のボットがコメントでプレビュー URL（「Visit Preview」のリンク）を教えてくれます。確認できたら Pull Request を `main` にマージすると、今度は Production デプロイが始まります。

> **つまずきポイント:** プレビュー URL を友達に送ったら、「Vercel のログイン画面が出て見られない」と言われた——これは故障ではありません。新しいプロジェクトでは、**Deployment Protection（デプロイの保護）**の「Standard Protection」が最初から有効になっていて、本番用の URL 以外（プレビュー URL と、ランダムな英数字の入った個別のデプロイ URL）は、Vercel にログインした権限のある人しか開けないようになっています。

- **NG:** プレビュー URL や長いデプロイ URL を、そのまま外部の人に送る（相手にはログイン画面が出る）
- **OK:** 一般の人に見せるのは、本番用の短い URL（`book-management-app.vercel.app`）にする。完成前のプレビューをどうしても見せたいときは、デプロイ画面の共有（Share）から作れる共有用リンクを使う
- 保護の設定は、プロジェクトの「Settings」→ サイドバーの「Deployment Protection」で確認できます。安全のため、学習中はオンのままにしておくのがおすすめです

> **注意:** Preview でも、環境変数に本番と同じ Supabase の URL とキーを入れていれば、**同じデータベース**につながります。プレビューで「削除」を試すと、本番のデータも消えます。本番のデータを守りたい場合は、Supabase で練習用のプロジェクトをもう1つ作り、その URL とキーを「Preview」だけにチェックして登録します（2.7 の手順で、Production 用と Preview 用に別々の値を登録できます）。

2.7 環境変数の追加・変更方法（デプロイ後）

デプロイ後に環境変数を変更したい場合:

1. Vercel ダッシュボードでプロジェクトを選択
2. 「Settings」を開き、左のサイドバーから「Environment Variables」を選ぶ（2026年の画面では、上のタブではなくサイドバーから選びます）
3. **Type** で「Config」か「Secret」を選び、Key・Value・Environments を入力して「Save」を押す。登録済みの変数を直すときは、その行の右端の「…」→「Edit」
4. **重要**: 環境変数を変更した後は、サイドバーの「Deployments」で最新のデプロイの「…」→「Redeploy」を選び、確認画面でもう一度「Redeploy」を押す必要があります

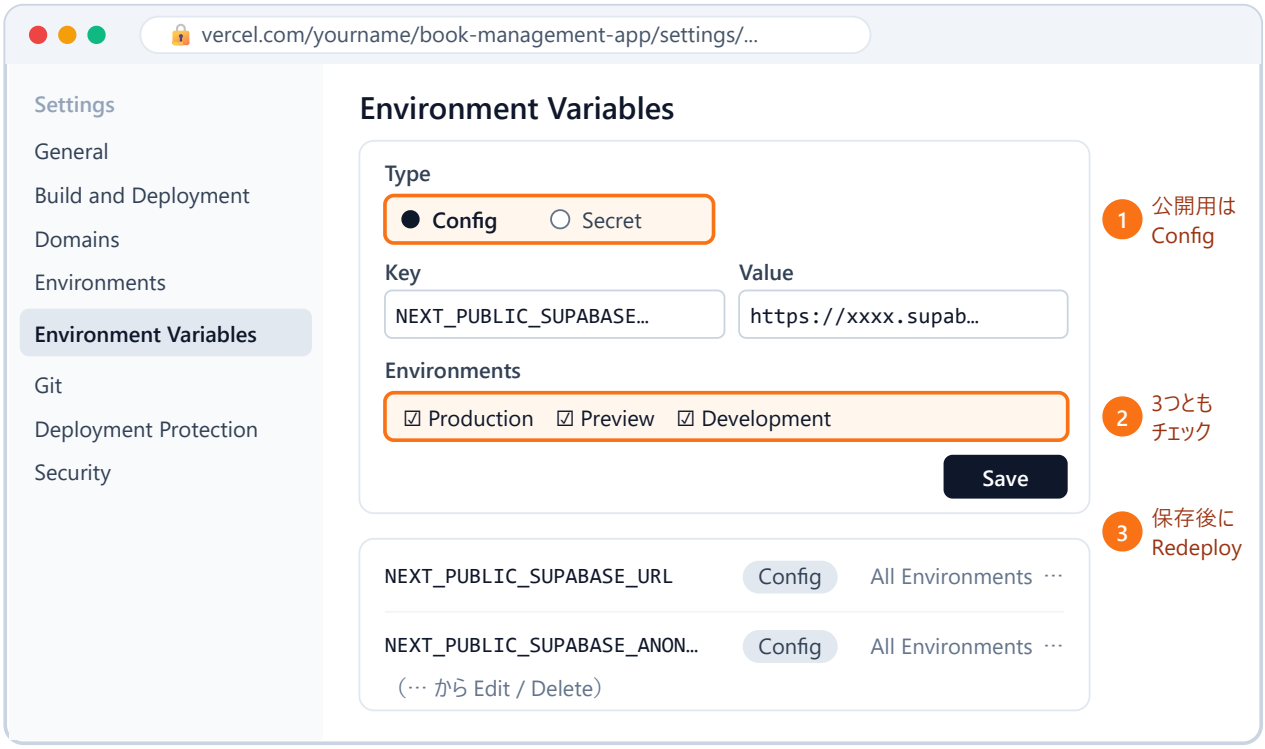


図10-12: 画面イメージ — プロジェクトの Settings → Environment Variables（2026年8月から Type に Config / Secret を選ぶ形式）

2026年8月から、環境変数を登録するときに **Type（種類）** を選ぶようになりました（それまでの「Sensitive」というスイッチの代わりです）。

Type	保存したあとに値を画面で見られるか	向いている値
Config（コンフィグ：設定値）	見られる	`NEXT_PUBLIC_SUPABASE_URL`、`NEXT_PUBLIC_SUPABASE_ANON_KEY` のような、公開してよい値
Secret（シークレット：秘密）	見られない（上書きはできる）	Secret key（`sb_secret_...`）など、本当に秘密の値

- `NEXT_PUBLIC_` 付きの値は、どのみちブラウザに配られる公開情報なので **Config** で問題ありません。あとで「何を登録したっけ？」と確認できて便利です。
- Redeploy の確認画面には「Use existing Build Cache（前回のビルド結果の一部を使い回す）」というチェックがあります。環境変数を変えたときは、念のためチェックを外して、最初からビルドし直すと確実です。

> **再デプロイが必要な理由（もう一度）**：Next.js は `NEXT_PUBLIC_` 付きの環境変数を「ビルド時」にコード内に埋め込みます。値を変えただけではコード内の値は古いままなので、もう一度ビルドし直す必要があるのです。Redeploy ボタンを押すと、Vercel は最新コミットを使って再ビルドし、新しい環境変数の値で動くバージョンを公開します。

2.8 カスタムドメインの設定（任意）

独自ドメイン（例: `mybooks.example.com`）を設定したい場合:

1. Vercel ダッシュボードでプロジェクトを選択
2. 「Settings」タブ → 「Domains」
3. ドメイン名を入力して「Add」
4. 表示される DNS 設定をドメインレジストラ（お名前.com、Cloudflare、Squarespace Domains（旧 Google Domains）など）で設定
5. DNS の反映を待つ（通常数分〜最大48時間）

> **初心者へ**: カスタムドメインは必須ではありません。Vercel が自動生成する `xxx.vercel.app` の URL でも十分に利用できます。DNS（ドメインネームシステム）は、世界中のサーバーに新しい設定が伝わるまで時間がかかります。設定した直後に反映されなくても焦らず待ちましょう。

2.9 デプロイ後の動作確認

デプロイが完了したら、以下の項目を確認しましょう。

- [ ] トップページが正常に表示される
- [ ] 書籍一覧が Supabase から取得されて表示される
- [ ] 新しい書籍を追加できる
- [ ] 書籍の詳細ページが表示される
- [ ] 書籍の情報を編集できる
- [ ] 書籍を削除できる
- [ ] スマートフォンでも正常に表示される（レスポンシブ）
- [ ] HTTPS（鍵アイコン）で接続されている
- [ ] `/api/health` が `"environment":"production"` と `true` を返す（2.9.1）

> **トラブルシューティング**: 問題が発生した場合は、Vercel ダッシュボードのサイドバーの「Deployments」から該当のデプロイを開くと、Deployment Details の「Building」の欄でビルドログを確認できます。ランタイムログ（アクセスされたときのログ）は、サイドバーの「Logs」で見られます。ビルドログは「ビルド時のエラー」、ランタイムログは「ユーザーアクセス時のエラー」を見るのに使います。

2.9.1 ヘルスチェック用の API で「どの版が、どの設定で動いているか」を確かめる

本番で何かおかしいとき、最初に知りたいのは「いま動いているのは、どのコミットから作られた版か」「環境変数はちゃんと入っているか」です。これを1回のアクセスで確かめられる、小さな API（ヘルスチェック：健康診断）を作っておきましょう。

> **▼ このコードがやること（先に日本語で）**： `/api/health` という URL を開くと、「アプリが動いているか」「本番かプレビューか」「どのコミットの版か」「Supabase の環境変数が登録されているか（true / false だけ）」を JSON で返します。値そのものは返さないので、本番に置いたままでも安全です。

コピー

```
// app/api/health/route.ts
// /api/health にアクセスすると「いま動いているアプリの状態」を JSON で返す
export function GET() {
  return Response.json({
    // ここまで処理が来れば、アプリ自体は動いている
    status: "ok",
    // Vercel 上なら production / preview、手元なら local
    environment: process.env.VERCEL_ENV ?? "local",
    // どのコミットから作られたか（先頭7文字）。手元なら local
    commit: process.env.VERCEL_GIT_COMMIT_SHA?.slice(0, 7) ?? "local",
    // 環境変数が「登録されているか」だけを true / false で返す（値そのものは返さない）
    supabaseUrlConfigured: Boolean(process.env.NEXT_PUBLIC_SUPABASE_URL),
    supabaseKeyConfigured: Boolean(process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY),
    // このレスポンスを作った時刻
    checkedAt: new Date().toISOString(),
  });
}
```

コピー

```
# ビルドして本番モードで起動したあと、別の PowerShell で実行する
curl.exe -s http://localhost:3001/api/health
```

▼ **実行結果**（手元。`.env.local` に2つの値が入っている場合）

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
{"status":"ok","environment":"local","commit":"local","supabaseUrlConfigured":true,"supabaseKeyConfigured":
true,"checkedAt":"2026-10-17T02:30:08.709Z"}
```

Vercel にデプロイしたあとは、ブラウザで `https://book-management-app.vercel.app/api/health` を開きます。たとえば次のように、`environment` が `production` に、`commit` がそのデプロイのコミット番号に変わります（値は一例です）。

コピー

```
{"status":"ok","environment":"production","commit":"a0b5a74","supabaseUrlConfigured":true,"supabaseKeyConfigured":true,"checkedAt":"..."}
```

▼ コードを1つずつ分解して解説

解説1: `route.ts` と `GET` 関数

コピー

```
export function GET() {
  return Response.json({ ... });
}
```

- `app/api/health/route.ts` という場所に置いたファイルは、画面（ページ）ではなく **API**（データだけを返す窓口）になります。URL は `/api/health` です。
- `GET` という名前で関数を export すると、ブラウザでその URL を開いたとき（GET リクエスト）に実行されます。`Response.json(...)` は「この中身を JSON 形式で返す」という意味です。

> **用語:** JSON（ジェイソン）… `{"名前": 値}` の形でデータを書く、Web でいちばんよく使われるデータの書き方。

解説2: Vercel が自動で入れてくれる環境変数

コピー

```
environment: process.env.VERCEL_ENV ?? "local",
commit: process.env.VERCEL_GIT_COMMIT_SHA?.slice(0, 7) ?? "local",
```

- `VERCEL_ENV` と `VERCEL_GIT_COMMIT_SHA` は、自分で登録しなくても Vercel が自動で用意する「システム環境変数」です。`VERCEL_ENV` は `production` / `preview` / `development` のどれか、`VERCEL_GIT_COMMIT_SHA` はデプロイのもとになったコミットの番号（40文字）が入ります。Environment Variables 画面の「Enable access to System Environment Variables」にチェックが入っていると使えます（通常は最初から入っています）。
- 手元にはこれらが無いので、`??`（左が無ければ右を使う）で `"local"` を返しています。`?.slice(0, 7)` は「値があれば先頭7文字だけ取り出す」という意味で、`git log --oneline` に出る短い番号と同じ長さです。

解説3: 値ではなく「あるかないか」だけを返す

コピー

```
supabaseUrlConfigured: Boolean(process.env.NEXT_PUBLIC_SUPABASE_URL),
```

- `Boolean(...)` は、値が入っていれば `true`、無ければ `false` に変換します。誰でも開ける URL なので、**値そのものは返さない**のがポイントです。
- これで、「本番で `false` になっている → Vercel に登録し忘れている」とすぐに分かります。

2.9.2 本番だけ動かないときの切り分け

「手元では動くのに、本番では動かない」ときは、やみくもに直さず、**どのログを見ればよいか**を先に決めます。次の3つの質問で、見る場所が決まります。

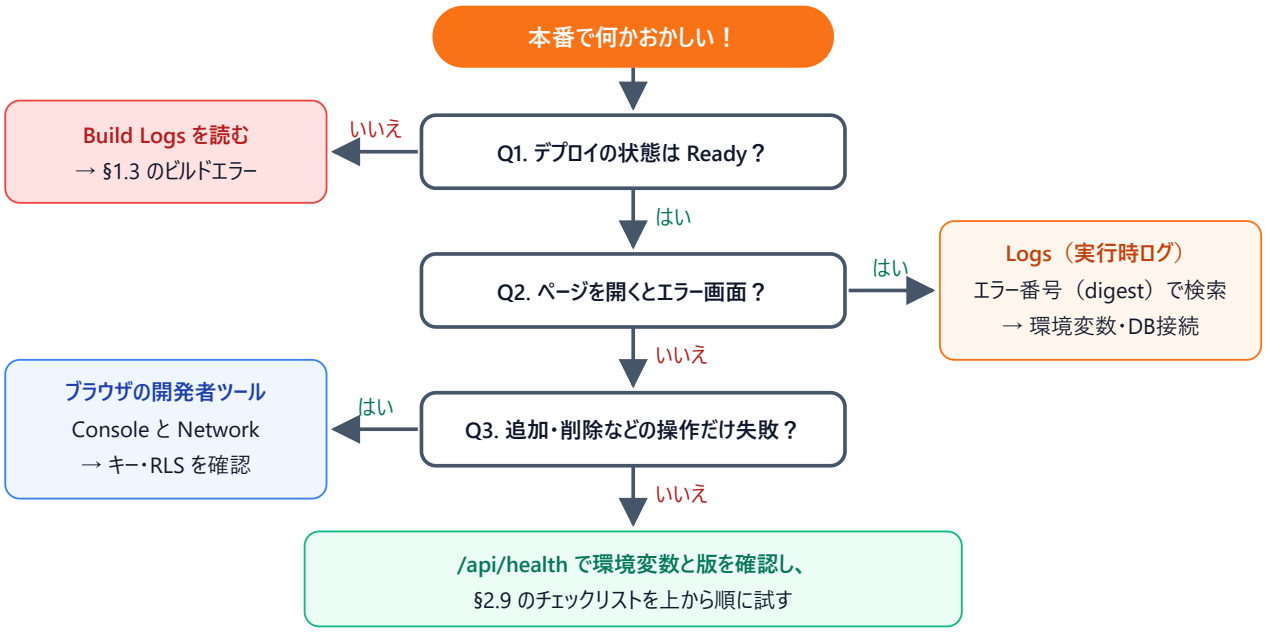


図10-13: 本番だけ動かないときの切り分け（どのログを見ればよいかを3つの質問で決める）

いちばん多い「環境変数の登録し忘れ」を、手元でわざと再現してみましょう。`.env.local` を一時的に別の名前に変え（`prebuild` のチェックも外した状態で）、ビルドして本番モードで動かします。

コピー

```
# .env.local が無い状態でビルドする（この教材のアプリはビルド自体は成功してしまう）
npm run build
npm run start -- -p 3001
# 別の PowerShell で、トップページの HTTP ステータスコードだけを表示する
curl.exe -s -o NUL -w "%{http_code}" http://localhost:3001/
# ヘルスチェックも確かめる
curl.exe -s http://localhost:3001/api/health
```

▼ 実行結果（`curl.exe` の結果）

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
PS> curl.exe -s -o NUL -w "%{http_code}" http://localhost:3001/
500
PS> curl.exe -s http://localhost:3001/api/health
{"status":"ok","environment":"local","commit":"local","supabaseUrlConfigured":false,"supabaseKeyConfigured"
:false,"checkedAt":"2026-10-17T02:37:18.617Z"}
```

▼ 実行結果（`npm run start` を動かしているターミナル。ファイルの場所は一部を伏せています）

コピー

```
✓ Ready in 287ms
✓ Running next.config.ts took 75ms
× Error: Your project's URL and Key are required to create a Supabase client!

Check your Supabase project's API settings to find these values

https://supabase.com/dashboard/project/_/settings/api
    at <unknown> (https://supabase.com/dashboard/project/_/settings/api)
    at <unknown> (C:\...\book-management\.next\server\chunks\ssr\_0ayfz82odguq8._.js:352:25)
    at du (C:\...\book-management\.next\server\chunks\ssr\_0ayfz82odguq8._.js:356:6060)
    at async f (C:\...\book-management\.next\server\chunks\ssr\_06ce-r9tf-zzn._.js:1:2218) {
  digest: '2080180400'
}
```

- `500` は「サーバーの中でエラーが起きた」という HTTP のステータスコードです（正常なら `200`）。
- ヘルスチェックは動いていて、`supabaseUrlConfigured` と `supabaseKeyConfigured` が `false` です。原因が「環境変数が無いこと」だと一目で分かります。
- ターミナル（Vercel なら「Logs」）には、エラー文と `digest`（ダイジェスト：エラーの識別番号）が出ています。

このとき、利用者のブラウザには次のような画面が表示されます（Next.js 16.4.1 の実際のエラー画面です。パソコンの設定によっては白い背景で表示されます）。画面の一番下の番号と、ログの `digest` が同じ番号になっているのがポイントです。



図10-14: 環境変数を登録し忘れたときに本番で起きること（左は Next.js 16 の実際のエラー画面、右はサーバーログ）

> **ヒント:** 本番のエラー画面には、セキュリティのためにエラーの詳しい中身は表示されません。利用者から「ERROR 2080180400 と出た」と教えてもらえれば、Vercel の「Logs」でその番号を検索して、原因のログにたどり着けます。

もう1つ、「本番だけ動かない」原因としてよくあるのが、URL の書き方です。

> **つまずきポイント:** コードの中に `http://localhost:3000` と書いてしまうと、本番では動きません。`localhost` は「**このコンピュータ自身**」という意味なので、本番のサーバーや利用者のスマホから見ると、あなたのPCではなく自分自身を指してしまうからです。

NG: 自分のPCの住所を書いている

コピー

```
// NG: 本番では「利用者自身のコンピュータ」を見に行ってしまう
const res = await fetch("http://localhost:3000/api/health");
```

OK: ブラウザ側のコードなら、`/` から始まる書き方にする

コピー

```
// OK: 今開いているサイトと同じ住所（ドメイン）に問い合わせる
const res = await fetch("/api/health");
```

2.10 ロールバック（前のバージョンに戻す）

どれだけ気をつけても、本番にバグを出してしまうことはあります。そんなときに慌てて直そうとすると、焦りから別のバグを生みがちです。まずは**正常に動いていた前のバージョンに戻して**利用者への影響を止め、落ち着いてから原因を直す——これがロールバックの考え方です。

Vercel のデプロイは、新しく作っても古いものが消えずに残っています。本番 URL は「どのデプロイを表示するか」を指しているだけなので、**指す先を前のデプロイに付け替えれば一瞬で戻せます。**これを **Instant Rollback（インスタント・ロールバック）**と呼びます。

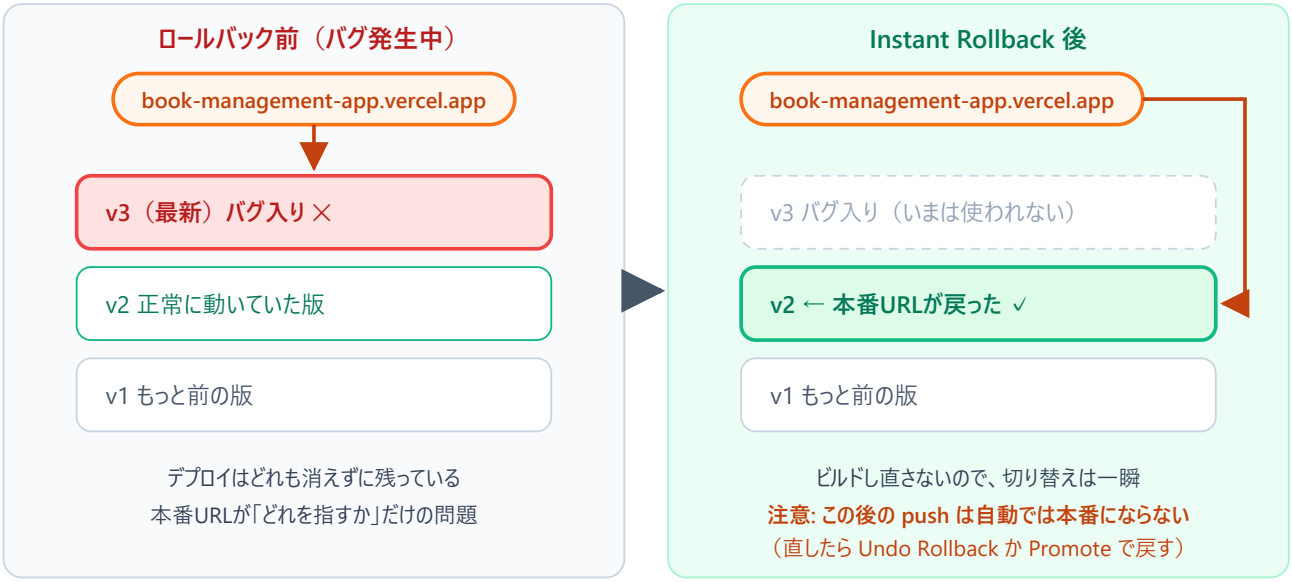


図10-15: Instant Rollback の仕組み（本番URLの「指し先」を前のデプロイに付け替えるだけ）

Instant Rollback の手順（ダッシュボード）

1. Vercel ダッシュボードでプロジェクトを開く
2. 概要（Overview）ページの「Production Deployment」の欄にある「**Instant Rollback**」を押す（サイドバーの「Deployments」で、戻したいデプロイの「…」から選ぶこともできます）
3. 現在の本番と、戻せるデプロイが表示されるので、戻したいデプロイを選んで「**Continue**」
4. 対象のドメインなどの確認内容を読んで「**Confirm Rollback**」
5. すぐに本番 URL が前のバージョンに切り替わる

- 無料の **Hobby プラン**では、戻せるのは「**直前の**本番デプロイ」だけです（Pro プラン以上なら、過去に本番だった好きなデプロイに戻せます）。
- ロールバックはビルドし直さないので、**環境変数は戻した先のデプロイを作ったときの値のまま**です。設定画面で値を変えていても反映されません。
- データベース（Supabase）の中身は戻りません。戻るのはアプリのプログラムだけです。

> **注意:** ロールバックすると、Vercel は「本番ドメインの自動の付け替え」を止めます。そのため、**ロールバックのあとに `main` へ push しても、自動では本番に出ません。**バグを直した版が Ready になったら、Production Deployment の欄に出る「**Undo Rollback**」を押して、本番にしたいデプロイを選んで「Confirm」します（Deployments の「…」→「Promote（本番に昇格させる）」でも同じです）。これで、push すると自動で本番に出る、いつもの状態に戻ります。

もう1つの戻し方: `git revert` で「取り消しのコミット」を作る

ダッシュボードを使わず、Git の履歴そのものを戻す方法もあります。`git revert` は、指定したコミットの変更を**打ち消す新しいコミット**を作るコマンドです。それを push すれば、Vercel が普通にビルドしてデプロイします（ビルドする分、Instant Rollback より時間はかかります）。

> **▼ このコードがやること（先に日本語で）:** 直前のコミット（バグを入れてしまったコミット）を取り消すコミットを作り、履歴を確認します。過去のコミットを消すのではなく「取り消しました」という記録を足すので、GitHub にすでに push した履歴でも安全に使えます。

コピー

```
# 直近3件のコミットを1行ずつ表示する
git log --oneline -3
# 直前のコミット（HEAD）を打ち消すコミットを作る（--no-edit はメッセージを自動で付ける）
git revert --no-edit HEAD
# もう一度、履歴を確かめる
git log --oneline -3
```

▼ 実行結果（実験用のリポジトリでの例）

コピー

```
PS> git log --oneline -3
649d244 トップページの見出しを変更（バグ入り）
f8386de 書籍詳細にメタデータを追加
8cb7ccb health + check-env
PS> git revert --no-edit HEAD
[main a0b5a74] Revert "トップページの見出しを変更（バグ入り）"
 Date: Sat Oct 17 11:31:49 2026 +0900
 1 file changed, 1 insertion(+), 1 deletion(-)
PS> git log --oneline -3
a0b5a74 Revert "トップページの見出しを変更（バグ入り）"
649d244 トップページの見出しを変更（バグ入り）
f8386de 書籍詳細にメタデータを追加
```

- `Revert "…"` という新しいコミット（`a0b5a74`）ができました。バグ入りのコミット（`649d244`）は履歴に残ったままですが、その変更内容はちょうど打ち消されています。
- あとは `git push` すれば、取り消し済みの版が Production デプロイとして公開されます。

戻し方	速さ	向いている場面
Instant Rollback	一瞬（ビルドしない）	今すぐ被害を止めたいとき。まずはこちら
`git revert` → push	1〜3分（ビルドする）	Git の履歴も「正しい状態」にそろえたいとき

> **ヒント:** ターミナル派の人は、Vercel CLI の `vercel rollback`（ロールバック）と `vercel promote`（本番に昇格）でも同じ操作ができます。

---

3. Supabase の本番設定

アプリが本番環境で公開された後は、セキュリティとデータ保護をしっかり確認する必要があります。

3.1 セキュリティの確認

API キーの種類を理解する

Supabase には2種類のキーがあります。2025年から新しい形式の **Publishable key**（`sb_publishable_...`）と **Secret key**（`sb_secret_...`）が導入され、従来の anon キー／service_role キーは2026年末に廃止予定の旧方式になりました（第5章）。役割は同じです。

キーの種類	用途	公開してよいか
Publishable key（旧 `anon` キー）	クライアントサイドからのアクセス	はい（RLS で保護）
Secret key（旧 `service_role` キー）	サーバーサイドからの管理アクセス	**絶対に公開してはいけない**

コピー

```
NEXT_PUBLIC_SUPABASE_URL          → クライアントに公開される（OK）
NEXT_PUBLIC_SUPABASE_ANON_KEY     → クライアントに公開される（OK、RLS が必須）
SUPABASE_SERVICE_ROLE_KEY         → サーバーサイドのみで使用（絶対に公開しない）
```

本番のアプリで、それぞれのキーがどこに置かれ、何がデータを守っているのかを図にすると、次のようになります。

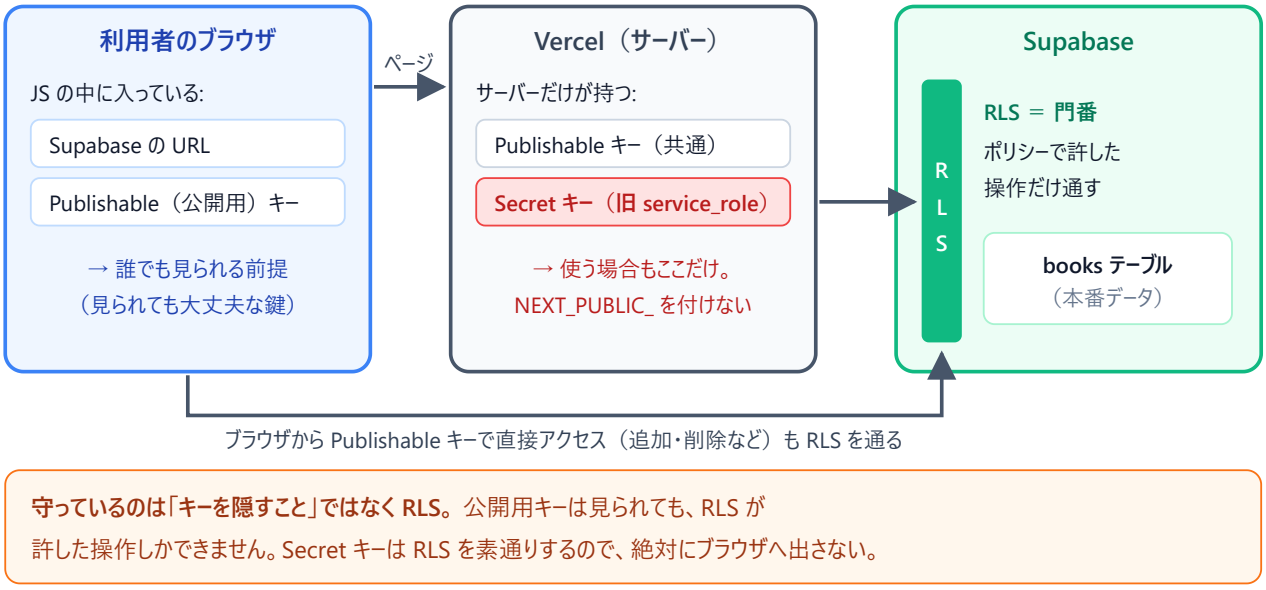


図10-16: 本番でのキーの置き場所と RLS の役割（公開用キーは見られる前提、守りの本体は RLS）

> **重要**: `NEXT_PUBLIC_` プレフィックスが付いた環境変数は、ブラウザの JavaScript から参照可能です。Secret key（旧 `service_role` キー）には絶対に `NEXT_PUBLIC_` を付けないでください。万一付けたままビルド・公開すると、悪意のあるユーザーがあなたのデータベースを全削除できてしまいます。

たとえば、`.env.local` や Vercel に次のように登録したとします。名前の付け方1つで、安全かどうかが決まります。

NG: 秘密のキーに `NEXT_PUBLIC_` を付けている

コピー

```
# NG: ブラウザ用の JS に値が埋め込まれ、世界中に公開される（0.3 の実験1 を参照）
NEXT_PUBLIC_SUPABASE_SECRET_KEY=sb_secret_xxxxxxxxxxxxxxx
```

OK: 秘密のキーには `NEXT_PUBLIC_` を付けない

コピー

```
# OK: サーバーで動くコードだけが読める（この教材のアプリでは、そもそも使わない）
SUPABASE_SECRET_KEY=sb_secret_xxxxxxxxxxxxxxx
```

> **つまずきポイント:** 「Secret key もブラウザから使えたら便利そう」と考えて `NEXT_PUBLIC_` を付けるのは、家の合鍵を玄関に貼り出すようなものです。もし付けたまま公開してしまったら、名前を直すだけでは足りません。Supabase の **Project Settings → API Keys** でそのキーを無効にして新しいキーを作り、新しい値を登録し直してください。

フロントエンドのコードで確認すべきこと

ブラウザの開発者ツール（F12 → Network タブ）で、以下を確認しましょう。

- Supabase への API リクエストが HTTPS で行われていること
- `service_role` キーがリクエストヘッダに含まれていないこと
- レスポンスに不要な個人情報が含まれていないこと

3.2 RLS（Row Level Security）ポリシーの見直し

RLS は Supabase のセキュリティの要です。本番環境では、適切なポリシーが設定されていることを必ず確認しましょう。

Auth Redirect URL の本番設定

将来的に認証機能を追加するときに必要な設定なので、ここで触れておきます。

1. Supabase ダッシュボード → Authentication → URL Configuration を開く
2. **Site URL** に本番 URL（例: `https://book-management-app.vercel.app`）を入れる
3. **Redirect URLs** に、ログイン後に戻ってきてよい URL を追加する（下の例を参照）

> **なぜ必要？:** Supabase Auth は「ログイン後にどの URL に戻していいか」を厳格に管理しています。ここに登録されていない URL に戻そうとすると、エラーで弾かれます。ローカル（ `http://localhost:3000` ）と本番（ `https://...vercel.app` ）の両方を登録しておくのが基本です。

登録する値の例です。`yourname` の部分は、あなたの Vercel のアカウント名（チーム名）に置き換えます。プレビュー URL の最後の `-yourname.vercel.app` の部分を見ると分かります。

コピー

```
Site URL:
https://book-management-app.vercel.app

Redirect URLs:
http://localhost:3000/**
https://book-management-app.vercel.app/**
https://*-yourname.vercel.app/**
```

- `*` は「`.` と `/` 以外の何文字でも」、`**` は「`/` も含めて何文字でも」に当てはまる記号です（Supabase 公式ドキュメントの説明）。
- 3行目の `https://*-yourname.vercel.app/**` は、`book-management-app-git-feature-review-yourname.vercel.app` のようなプレビュー URL に当てはまります。
- Site URL は、戻り先を指定しなかったときに使われる「基本の戻り先」です。初期値は `http://localhost:3000` なので、公開したら本番の URL に変えておきます。

> **つまずきポイント:** プレビュー用だからといって `https://*.vercel.app/**` のように広く登録してはいけません。これだと**他人が Vercel で作ったアプリ**まで「戻ってきてよい場所」に含まれてしまい、ログインの情報を横取りされる危険があります。必ず自分のアカウント名（ `-yourname` ）まで含めて登録しましょう。

現在の RLS 設定を確認する

Supabase ダッシュボード → Table Editor → `books` テーブル →「RLS」タブで確認できます。

この教材では、認証なしで誰でもアクセスできる設定にしています。

コピー

```
-- 現在のポリシー（開発用・学習用）
-- -- で始まる行は SQL のコメント（実行されない）
-- すべてのユーザーが読み取り可能
-- ポリシー名を「誰でも書籍を読める」とし、books テーブルに適用
CREATE POLICY "誰でも書籍を読める" ON books
  -- FOR SELECT = 読み取り操作に対する許可。USING (true) = 常に許可
  FOR SELECT USING (true);

-- すべてのユーザーが書き込み可能
CREATE POLICY "誰でも書籍を追加できる" ON books
  -- FOR INSERT = 追加操作。WITH CHECK (true) = 常に許可
  FOR INSERT WITH CHECK (true);

-- すべてのユーザーが更新可能
CREATE POLICY "誰でも書籍を更新できる" ON books
  -- FOR UPDATE = 更新操作。常に許可
  FOR UPDATE USING (true);

-- すべてのユーザーが削除可能
CREATE POLICY "誰でも書籍を削除できる" ON books
  -- FOR DELETE = 削除操作。常に許可
  FOR DELETE USING (true);
```

本番環境向けの推奨設定

本格的なアプリケーションでは、認証を導入した上で以下のようなポリシーに変更することを推奨します。

コピー

```
-- 本番用ポリシー（認証導入後）
-- 誰でも読み取り可能（公開データの場合）
CREATE POLICY "誰でも書籍を読める" ON books
  -- 読み取りは全員OK
  FOR SELECT USING (true);

-- 認証済みユーザーのみ書き込み可能
CREATE POLICY "認証済みユーザーのみ追加可能" ON books
  -- auth.role() = 現在のユーザーの権限を返す関数。'authenticated' = ログイン済み
  FOR INSERT WITH CHECK (auth.role() = 'authenticated');

-- 自分が追加した書籍のみ更新可能
CREATE POLICY "自分の書籍のみ更新可能" ON books
  -- auth.uid() = ログイン中のユーザーID。user_id 列と一致する行だけ更新可
  FOR UPDATE USING (auth.uid() = user_id);

-- 自分が追加した書籍のみ削除可能
CREATE POLICY "自分の書籍のみ削除可能" ON books
  -- 自分のレコードしか消せない（他人のは触れない）
  FOR DELETE USING (auth.uid() = user_id);
```

> **この教材の範囲**: 認証機能は次のステップとして紹介しますので、現時点では開発用のポリシーのままで問題ありません。ただし、**重要なデータを扱う本番アプリではセキュリティ設定を必ず強化してください。**

3.3 バックアップの設定

データベースのバックアップは、データ損失を防ぐために非常に重要です。

Supabase の自動バックアップ

- **Free プラン**: 自動バックアップなし（手動でのみ可能）
- **Pro プラン以上**: 毎日の自動バックアップ、Point-in-Time Recovery（PITR）

手動バックアップの方法

Supabase ダッシュボードから手動でバックアップを取得できます。

1. Supabase ダッシュボード → Settings → Database
2. 「Database Backups」セクション
3. 「Download backup」をクリック

または、`pg_dump` コマンドを使用する方法:

コピー

```
# Supabase のデータベース URL は、ダッシュボードの
# Settings → Database → Connection string で確認できます

# pg_dump = PostgreSQL に同梱されているバックアップ用コマンド（PostgreSQL クライアントのインストールが必要）
# "postgresql://..." = データベースへの接続文字列。"ユーザー名:パスワード@ホスト:ポート/データベース名" の形
# > backup.sql = 出力を backup.sql ファイルに書き込む（リダイレクト）。実行すると同じフォルダにバックアップが保存される
pg_dump "postgresql://postgres:[パスワード]@db.[プロジェクトID].supabase.co:5432/postgres" > backup.sql
```

> **初心者へ**: 学習目的であれば手動バックアップで十分です。本番アプリケーションを運用する際は、Pro プラン以上の利用を検討してください。

3.4 無料プランの「一時停止」に注意

公開してしばらくたったある日、「急に本が1冊も表示されなくなった」「登録ボタンを押しても失敗する」ということがあります。コードは何も変えていないのに、です。原因としてよくあるのが、**Supabase の無料プランのプロジェクトの一時停止**（**Pause**）です。

- Supabase は、Free プランのプロジェクトで**7日間ほとんど使われていない**と判断すると、プロジェクトを一時停止することがあります（データベースへのアクセスが少しでも毎日あれば、通常は止まりません）。
- 止まる約1週間前に警告のメールが、止まったあとに通知のメールが届きます。
- 一時停止中は、アプリからデータベースに一切つながりません。Vercel 側は動いているので、画面は出るのにデータが空になる、という症状になります。
- 再開するには、Supabase ダッシュボードでそのプロジェクトを開き、「Resume project」ボタンを押して確認します。データと設定は止まる前の状態のまま戻ります（停止から1年以内なら再開できます）。

> **つまずきポイント:** 「Vercel のデプロイが壊れた」と思って、コードを何度も直したり Redeploy したりしても直りません。データだけが出ないときは、Vercel より先に **Supabase のダッシュボードを開いて、プロジェクトが一時停止していないか**を確かめましょう。学習用のアプリなら、使うときに再開すれば十分です。毎日使う本番アプリなら、一時停止の無い有料の Pro プランを検討します。

---

4. パフォーマンス最適化

デプロイしたアプリのパフォーマンスを向上させるためのテクニックを紹介します。

4.1 Next.js の Image コンポーネント

Next.js には、画像を自動的に最適化する `Image` コンポーネントが用意されています。書籍のカバー画像などを扱う場合に効果的です。

通常の `<img>` タグとの違い

コピー

```
// ❌ 通常の img タグ（最適化なし）
// 単純な HTML タグ。画像の自動最適化やレイアウトずれ防止は行われない
<img src="/book-cover.jpg" alt="書籍カバー" width={200} height={300} />

// ✅ Next.js の Image コンポーネント（自動最適化）
// next/image からインポートして使う。デフォルトエクスポートを Image という名前で受け取る
import Image from 'next/image';

<Image
  // 画像のパス（public/ フォルダ基準）
  src="/book-cover.jpg"
  // 代替テキスト（必須。アクセシビリティ・SEO 向上）
  alt="書籍カバー"
  // 元画像の幅（px単位）。レイアウトずれ防止に必須
  width={200}
  // 元画像の高さ（px単位）。レイアウトずれ防止に必須
  height={300}
  // 読み込み中にぼかし表示。"blur" か "empty" を指定
  placeholder="blur"
  // ぼかし画像のデータURL。低解像度のプレースホルダ用
  blurDataURL="data:..."
/>
```

Image コンポーネントのメリット:

機能	説明
自動リサイズ	デバイスに合わせた最適なサイズで配信
形式変換	WebP/AVIF など効率的な形式に自動変換
遅延読み込み	ビューポートに入るまで読み込みを遅延
CLS 防止	width/height の指定でレイアウトシフトを防止
キャッシュ	最適化した画像をキャッシュして再利用

外部画像を使用する場合

外部サイトの画像を使用する場合は、`next.config.js` に許可するドメインを追加する必要があります。

コピー

```
// next.config.js
// JSDoc コメント。TypeScript の型情報を JS ファイルに付与する記法
/** @type {import('next').NextConfig} */
const nextConfig = {
  // images = next/image の設定
  images: {
    // remotePatterns = 外部画像を許可するドメインパターンのリスト
    // ここに登録されていないドメインの画像は <Image> で読み込めない（セキュリティのため）
    remotePatterns: [
      {
        // https のみ許可（http は不可）
        protocol: 'https',
        // 画像を取得するドメイン
        hostname: 'example.com',
        // /images/ 以下の任意のパスを許可（** はワイルドカード）
        pathname: '/images/**',
      },
      {
        protocol: 'https',
        // Supabase の任意のサブドメインを許可。* は単一階層のワイルドカード
        hostname: '*.supabase.co',
        // /storage/ 以下を許可
        pathname: '/storage/**',
      },
    ],
  },
};

// CommonJS の書き方で外部に公開
// Next.js はこのファイルを require して、ここから設定オブジェクトを読み取る
module.exports = nextConfig;
```

4.2 メタデータの設定（SEO）

検索エンジンにアプリを正しく認識してもらうために、メタデータを設定しましょう。

> **SEO とは？** Search Engine Optimization（検索エンジン最適化）の略。Google などの検索エンジンに、自分のサイトを正しく理解してもらい、検索結果に出やすくする工夫のこと。

ルートレイアウトでのメタデータ設定

コピー

```
// app/layout.tsx
// Metadata 型を next からインポート。TypeScript の型チェックに使う
import type { Metadata } from 'next';

// metadata という名前で export すると、Next.js が自動でメタタグを生成してくれる
// : Metadata は型注釈。間違ったキーを書くとエディタ・ビルド時にエラーになる
export const metadata: Metadata = {
  // title = <title> タグの内容を設定
  title: {
    // デフォルトのタイトル（titleが指定されていないページで使用）
    default: '書籍管理アプリ',
    // 各ページのタイトルが自動的に「ページ名 | 書籍管理アプリ」になる（%s が各ページのtitleで置き換わる）
    template: '%s | 書籍管理アプリ',
  },
  // <meta name="description"> 用
  description: 'お気に入りの書籍を管理・評価できるWebアプリケーションです。',
  // <meta name="keywords"> 用（現代では重要度は低い）
  keywords: ['書籍管理', '本', 'レビュー', 'Next.js'],
  // 著者情報
  authors: [{ name: 'あなたの名前' }],
  // openGraph = OGP（OpenGraph Protocol）の設定。SNS でシェアしたときの見た目に使われる
  openGraph: {
    title: '書籍管理アプリ',
    description: 'お気に入りの書籍を管理・評価できるWebアプリケーションです。',
    // サイトの代表URL
    url: 'https://your-app.vercel.app',
    // サイト名
    siteName: '書籍管理アプリ',
    // 言語と地域
    locale: 'ja_JP',
    // コンテンツ種別。website / article など
    type: 'website',
  },
  // twitter = X（旧Twitter）でシェアされたときのカード設定
  twitter: {
    // 大きな画像付きカードを使用
    card: 'summary_large_image',
    title: '書籍管理アプリ',
    description: 'お気に入りの書籍を管理・評価できるWebアプリケーションです。',
  },
  // robots = 検索エンジンクローラへの指示
  robots: {
    // true = 検索結果に表示してOK
    index: true,
    // true = ページ内のリンクをたどってOK
    follow: true,
  },
};
```

各ページでのメタデータ設定

コピー

```
// app/page.tsx（書籍一覧ページ）
import type { Metadata } from 'next';

// このページ固有のメタデータを定義
export const metadata: Metadata = {
  // → 「書籍一覧 | 書籍管理アプリ」と表示される（layout.tsx の template が適用される）
  title: '書籍一覧',
  description: '登録されている書籍の一覧を表示します。',
};
```

動的なメタデータ（書籍詳細ページ）

コピー

```
// app/books/[id]/page.tsx（generateMetadata の部分だけの抜粋。ページ本体の関数は第8章のまま）
import type { Metadata } from "next";
// サーバー用の Supabase クライアント（第7章で作ったもの）
import { createClient } from "@/lib/supabase/server";

// このページに渡される props の型
type Props = {
  // URL の [id] 部分。Next.js 15 以降は Promise（あとから値が届く箱）で渡される
  params: Promise<{ id: string }>;
};

// generateMetadata = 動的にメタデータを作る関数。Next.js が自動で呼び出す
// async = 中で await（非同期処理）を使えるようにする
// Promise<Metadata> = 「Metadata を返す非同期関数」という型注釈
export async function generateMetadata({ params }: Props): Promise<Metadata> {
  // await で箱の中身を取り出してから id を使う
  const { id } = await params;
  // サーバー用クライアントを作る（cookies() を使うので await が必要）
  const supabase = await createClient();
  // id が一致する書籍を1件だけ取得する（.single() = 1件だけ取得）
  const { data: book } = await supabase
    .from("books")
    .select("*")
    .eq("id", id)
    .single();

  // 本が見つからなかったときの表示
  if (!book) {
    return { title: "書籍が見つかりません" };
  }

  // 見つかったときは、タイトルと著者名を使ってメタデータを作る
  return {
    title: book.title,
    description: `${book.title}（${book.author}）の詳細情報`,
  };
}
```

> **注意:** 古い記事では `params: { id: string }` と書いて `params.id` を直接使っていますが、Next.js 15 以降では `params` は Promise なので、必ず `await params` で取り出します（1.3 のエラー6 を参照）。このコードは Next.js 16.4.1 でビルドが通ることを確認しています。

4.3 Lighthouse での計測方法

Google Lighthouse は、Web アプリのパフォーマンス、アクセシビリティ、SEO などを総合的に評価するツールです。

Lighthouse の使い方

1. Chrome ブラウザでデプロイしたアプリを開く
2. F12 キーで開発者ツールを開く
3. 上部タブから「Lighthouse」を選択
4. 「Analyze page load」をクリック
5. 数十秒後に結果が表示される

評価項目

項目	説明	目標スコア
Performance	ページの読み込み速度	90 以上
Accessibility	アクセシビリティ（音声読み上げ対応など）	90 以上
Best Practices	Web 開発のベストプラクティス準拠	90 以上
SEO	検索エンジン最適化	90 以上

スコアを上げるコツ

- **Performance**: Image コンポーネントの使用、不要な JavaScript の削除、フォントの最適化
- **Accessibility**: alt 属性の設定、十分なカラーコントラスト、セマンティックな HTML
- **Best Practices**: HTTPS の使用、コンソールエラーの解消
- **SEO**: メタデータの設定、title タグの設定、レスポンシブデザイン

---

5. 学習の振り返り

ここまでお疲れさまでした。この教材を通じて、あなたは多くのことを学びました。一つずつ振り返ってみましょう。

5.1 この教材で学んだこと

以下のチェックリストで、あなたの学習成果を確認してください。

TypeScript の基礎

- [x] 変数の型注釈（`string`, `number`, `boolean`）
- [x] 型エイリアス（`type Book = { ... }`）
- [x] ジェネリクスの基本的な使い方
- [x] `interface` と `type` の違い
- [x] オプショナルプロパティ（`?`）

- [x] 型推論の仕組み

React の基礎

- [x] コンポーネントとは何か
- [x] JSX の書き方
- [x] Props の受け渡し
- [x] State の管理（`useState`）
- [x] 副作用の処理（`useEffect`）
- [x] イベントハンドリング（`onClick`, `onChange`, `onSubmit`）
- [x] 条件付きレンダリング
- [x] リストのレンダリング（`map` と `key`）
- [x] フォームの作成と制御

Next.js の基礎

- [x] App Router の仕組み（ファイルベースルーティング）
- [x] Server Components と Client Components の違い
- [x] `'use client'` ディレクティブの使い方
- [x] `page.tsx` と `layout.tsx` の役割
- [x] 動的ルート（`[id]`）
- [x] リンクとナビゲーション（`Link` コンポーネント）
- [x] メタデータの設定

Supabase でのデータベース操作

- [x] Supabase プロジェクトの作成
- [x] テーブルの作成
- [x] RLS（Row Level Security）の設定
- [x] データの取得（`SELECT`）
- [x] データの追加（`INSERT`）
- [x] データの更新（`UPDATE`）
- [x] データの削除（`DELETE`）
- [x] Supabase クライアントの初期化

CRUD アプリケーションの実装

- [x] **C**reate: 新しい書籍の追加フォーム
- [x] **R**ead: 書籍一覧の表示、詳細ページ
- [x] **U**pdate: 書籍情報の編集フォーム

- [x] Delete: 書籍の削除（確認ダイアログ付き）
- [x] ローディング状態の処理
- [x] エラーハンドリング

Tailwind CSS でのスタイリング

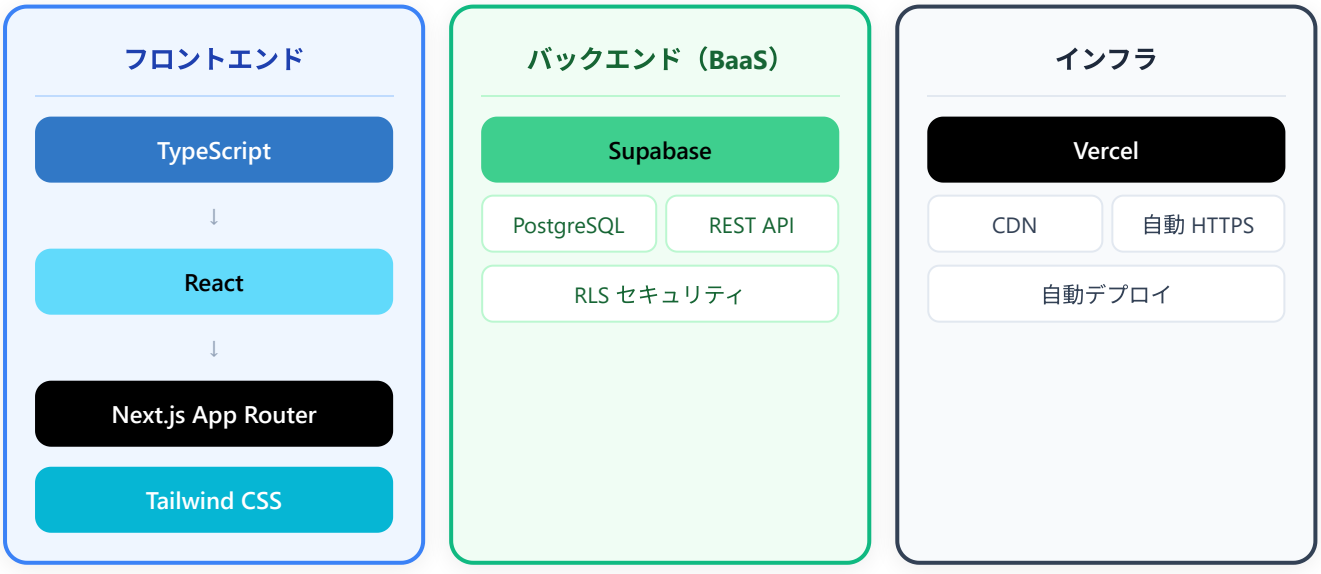
- [x] ユーティリティクラスの基本
- [x] レスポンシブデザイン（`sm:`, `md:`, `lg:`）
- [x] Flexbox / Grid レイアウト
- [x] ホバー・フォーカスのスタイル
- [x] カラーパレットの使い方
- [x] カスタムコンポーネントのスタイリング

Vercel へのデプロイ

- [x] GitHub リポジトリの作成
- [x] Vercel アカウントの作成と連携
- [x] 環境変数の設定
- [x] 自動デプロイの仕組み
- [x] デプロイ後の動作確認
- [x] ビルドエラーの読み方と直し方
- [x] Preview と Production の違い
- [x] ロールバック（前のバージョンに戻す）

5.2 技術スタック全体像

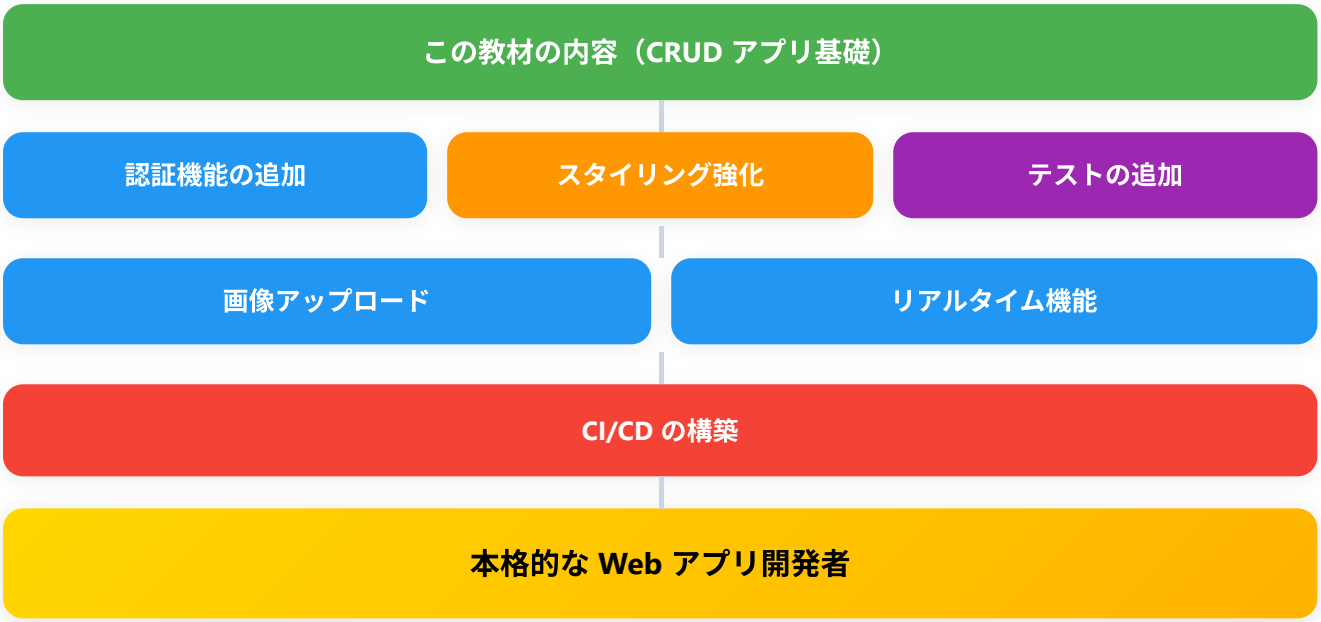
この教材で使用した技術スタックを図で確認しましょう。



6. 次のステップ（発展学習）

この教材で基礎を身につけたあなたは、さらに多くの機能を追加できます。以下に、次のステップとして取り組める発展的なトピックを紹介します。

6.1 学習ロードマップ



6.2 認証機能の追加（Supabase Auth）

ユーザーがアカウントを作成し、ログインしてから書籍を管理できるようにします。

実装のイメージ:

コピー

```
// Supabase Auth を使ったログイン例
// supabase クライアントをインポート
import { supabase } from '@/lib/supabase';

// メールアドレス + パスワードでサインアップ（新規登録）
// supabase.auth.signUp = 新規ユーザー作成のメソッド。Promise を返すので await で待つ
// 戻り値の data には作成されたユーザー情報、error には失敗時の情報が入る
const { data, error } = await supabase.auth.signUp({
  // 登録するメールアドレス
  email: 'user@example.com',
  // パスワード（Supabase の最小文字数ポリシーを満たす必要あり）
  password: 'your-password',
});

// ログイン
// signInWithPassword = メール・パスワードでログインするメソッド
const { data, error } = await supabase.auth.signInWithPassword({
  email: 'user@example.com',
  password: 'your-password',
});

// Google ログイン
// signInWithOAuth = 外部プロバイダ（Google, GitHub 等）でログインするメソッド
// 内部的にプロバイダの認証画面にリダイレクトされる
const { data, error } = await supabase.auth.signInWithOAuth({
  // 認証プロバイダの種類
  provider: 'google',
});

// ログアウト
// signOut = 現在のセッションを破棄するメソッド
const { error } = await supabase.auth.signOut();
```

学べること:

- ユーザー認証の仕組み
- セッション管理
- 保護されたルート（ログインしていないとアクセスできないページ）
- RLS ポリシーとの連携

参考リソース:

- Supabase Auth ドキュメント
- Next.js + Supabase Auth ガイド

6.3 画像アップロード（Supabase Storage）

書籍のカバー画像をアップロードして表示する機能を追加します。

実装のイメージ:

コピー

```
// Supabase Storage を使った画像アップロード例
import { supabase } from '@/lib/supabase';

// 画像のアップロード
// supabase.storage.from('バケット名') = 指定したバケット（保存場所）を操作する
// .upload(パス, ファイル, オプション) = ファイルをアップロードする
const { data, error } = await supabase.storage
  // バケット名（事前に Supabase ダッシュボードで作成しておく）
  .from('book-covers')
  // covers/ 以下に fileName でアップロード
  .upload(`covers/${fileName}`, file, {
    // ブラウザにキャッシュさせる秒数（3600秒 = 1時間）
    cacheControl: '3600',
    // false = 同名ファイルがあればエラー。true なら上書き
    upsert: false,
  });

// アップロードした画像の公開 URL を取得
// getPublicUrl = バケットが Public 設定の場合に公開URLを取得する
const { data: { publicUrl } } = supabase.storage
  // バケット名
  .from('book-covers')
  // 取得したいファイルのパス
  .getPublicUrl(`covers/${fileName}`);
```

学べること:

- ファイルアップロードの仕組み
- Supabase Storage の使い方
- 画像のプレビュー表示
- ファイルサイズ・形式のバリデーション

参考リソース:

- Supabase Storage ドキュメント

6.4 リアルタイム機能（Supabase Realtime）

複数ユーザーが同時にアプリを開いているとき、誰かが書籍を追加・編集すると、他のユーザーの画面にもリアルタイムで反映される機能です。

実装のイメージ:

コピー

```
// Supabase Realtime を使ったリアルタイム購読例
// useEffect を使うので Client Component にする必要がある
'use client';

import { useEffect } from 'react';
import { supabase } from '@/lib/supabase';

// ページコンポーネント。default export で Next.js のルーティング対象にする
export default function BooksPage() {
  // useEffect = コンポーネントのマウント時に副作用を実行するフック
  // 第2引数 [] により、初回マウント時に1度だけ実行される
  useEffect(() => {
    // books テーブルの変更をリアルタイムで監視
    // .channel('チャンネル名') = WebSocket チャンネルを作成
    const channel = supabase
      // チャンネルの名前。任意の文字列
      .channel('books-changes')
      .on(
        // PostgreSQL の変更イベントを監視
        'postgres_changes',
        {
          // INSERT, UPDATE, DELETE すべて（'*' = 全種類）
          event: '*',
          // スキーマ名（Supabase のデフォルトは 'public'）
          schema: 'public',
          // 監視対象のテーブル名
          table: 'books',
        },
        (payload) => {
          // 変更が起きるたびに呼ばれるコールバック関数
          // payload には変更内容が入っている
          console.log('変更を検知:', payload);
          // ここで State を更新してUIに反映
        }
      )
      // 監視開始
      .subscribe();

    // クリーンアップ
    // useEffect が return する関数はコンポーネントのアンマウント時に呼ばれる
    return () => {
      // チャンネルを削除して購読停止（メモリリーク防止）
      supabase.removeChannel(channel);
    };
  // 空配列 = 初回1回だけ実行
  }, []);
```

```
  return <div>...</div>;
}
```

学べること:

- WebSocket の仕組み
- リアルタイムデータ同期
- 楽観的更新（Optimistic Update）

参考リソース:

- Supabase Realtime ドキュメント

6.5 テストの追加（Jest, React Testing Library）

コードの品質を保証するために、テストを書く方法を学びます。

実装のイメージ:

コピー

```
// BookCard コンポーネントのテスト例
// render = コンポーネントをテスト用の仮想DOMに描画する関数
// screen = 描画された要素を取得するためのオブジェクト
import { render, screen } from '@testing-library/react';
import BookCard from '@/components/BookCard';

// describe = 関連するテストをグループ化するブロック
// 第1引数: グループ名、第2引数: 中で it() を並べる関数
describe('BookCard', () => {
  // テスト用のダミーデータ
  const mockBook = {
    id: '1',
    title: '吾輩は猫である',
    author: '夏目漱石',
    rating: 5,
    created_at: '2025-01-01',
  };

  // it = 個別のテストケース。test() でも同じ
  // 第1引数: テストの説明、第2引数: 実行する関数
  it('書籍のタイトルが表示される', () => {
    // コンポーネントを描画
    render(<BookCard book={mockBook} />);
    // expect(...).toBeInTheDocument() = 要素が画面に存在することを検証
    // screen.getByText('文字列') = その文字列を含む要素を取得
    expect(screen.getByText('吾輩は猫である')).toBeInTheDocument();
  });

  it('著者名が表示される', () => {
    render(<BookCard book={mockBook} />);
    expect(screen.getByText('夏目漱石')).toBeInTheDocument();
  });

  it('評価が星で表示される', () => {
    render(<BookCard book={mockBook} />);
    // getAllByText = 該当する全要素を配列で取得
    const stars = screen.getAllByText('★');
    // toHaveLength(5) = 配列の長さが5であることを検証
    expect(stars).toHaveLength(5);
  });
});
```

学べること:

- ユニットテスト・統合テストの考え方
- Jest テストランナーの使い方

- React Testing Library でのコンポーネントテスト
- テスト駆動開発（TDD）の基礎

参考リソース:

- Jest 公式ドキュメント
- React Testing Library 公式ドキュメント

6.6 CI/CD の構築

GitHub Actions を使って、コードをプッシュするたびに自動でテストを実行し、品質を保つ仕組みを構築します。

実装のイメージ:

コピー

```yaml
# .github/workflows/ci.yml
# このファイルを置くだけで GitHub Actions が自動で読み込んでくれる
# YAML はインデント（半角スペース2つ）で階層を表す形式

# name = ワークフローの名前（GitHub の Actions タブに表示される）
name: CI

# on = いつこのワークフローを起動するかの条件
on:
  push:
    # main ブランチに push されたとき
    branches: [main]
  pull_request:
    # main ブランチ向けの Pull Request が作られたとき
    branches: [main]

# jobs = 実行するジョブの定義（複数並列に書ける）
jobs:
  # ジョブの名前（任意）
  test:
    # 実行環境。最新の Ubuntu Linux を使う
    runs-on: ubuntu-latest
    # 順に実行するステップを並べる
    steps:
      # リポジトリのコードを取得する公式アクション
      - uses: actions/checkout@v4
      # Node.js をセットアップする公式アクション
      - uses: actions/setup-node@v4
        with:
          # Node.js のバージョン
          node-version: '20'
          # npm のキャッシュを使って高速化
          cache: 'npm'

      # npm ci = package-lock.json に従ってクリーンインストール
      - run: npm ci
      # lint チェックを実行
      - run: npm run lint
      # ビルドを実行（型エラーがないか確認）
      - run: npm run build
      # テストを実行
      - run: npm test
```

学べること:

- CI/CD の概念と重要性
- GitHub Actions の使い方

- 自動テスト・自動デプロイのパイプライン
- ブランチ戦略（main, develop, feature ブランチ）

参考リソース:

- GitHub Actions 公式ドキュメント

---

7. 推奨学習リソース

7.1 公式ドキュメント

リソース名	URL	説明
Next.js 公式ドキュメント	https://nextjs.org/docs	Next.js のすべての機能を網羅。チュートリアルも充実
React 公式ドキュメント	https://ja.react.dev	React の基礎から応用まで。日本語版あり
TypeScript 公式ドキュメント	https://www.typescriptlang.org/docs/	TypeScript の型システムを深く学べる
Supabase 公式ドキュメント	https://supabase.com/docs	Supabase の全機能のガイド
Tailwind CSS 公式ドキュメント	https://tailwindcss.com/docs	ユーティリティクラスの全リファレンス
Vercel 公式ドキュメント	https://vercel.com/docs	デプロイ・設定の詳細ガイド
MDN Web Docs	https://developer.mozilla.org/ja/	Web 技術全般のリファレンス。日本語版あり

7.2 おすすめの書籍

書籍名	対象	概要
『プロを目指す人のためのTypeScript入門』	初級〜中級	TypeScript を体系的に学べる定番書
『React ハンズオンラーニング 第2版』	初級〜中級	React の基礎を手を動かしながら学べる
『実践 Next.js』	中級	App Router を含む Next.js の実践的な解説
『Web API: The Good Parts』	中級	API 設計の基礎を学べる
『りあクト! TypeScript で始めるつらくない React 開発』	初級	React + TypeScript を優しく解説

7.3 コミュニティ

コミュニティ	URL	特徴
Zenn	https://zenn.dev	日本語の技術記事プラットフォーム。Next.js 関連の記事が豊富
Qiita	https://qiita.com	日本最大級の技術情報共有サービス
Stack Overflow	https://stackoverflow.com	世界最大の Q&A サイト。英語だが情報量が圧倒的
Discord（各技術の公式サーバー）	-	Next.js、Supabase 等それぞれ公式 Discord がある
X（旧 Twitter）	https://x.com	`#nextjs` `#supabase` などのハッシュタグで最新情報をキャッチ

---

8. 付録: 全ファイルの完成版一覧

この教材で作成したすべてのファイルの一覧です。

8.1 プロジェクト構成

コピー

```
book-management-app/
├── app/
│   ├── layout.tsx
│   ├── page.tsx
│   ├── globals.css
│   ├── loading.tsx
│   ├── error.tsx
│   ├── api/
│   │   └── health/
│   │       └── route.ts
│   └── books/
│       ├── new/
│       │   └── page.tsx
│       └── [id]/
│           ├── page.tsx
│           ├── not-found.tsx
│           └── edit/
│               └── page.tsx
├── components/
│   ├── Header.tsx
│   ├── BookCard.tsx
│   ├── BookList.tsx
│   ├── BookForm.tsx
│   ├── DeleteButton.tsx
│   ├── StatusBadge.tsx
│   ├── RatingStars.tsx
│   ├── LoadingSpinner.tsx
│   ├── SearchBar.tsx
│   └── SortSelect.tsx
├── lib/
│   └── supabase/
│       ├── client.ts
│       └── server.ts
├── types/
│   └── book.ts
├── public/
│   └── favicon.ico
├── scripts/
│   └── check-env.mjs
├── .env.local
├── .gitignore
├── eslint.config.mjs
├── next.config.ts
├── package.json
```

```
├── tsconfig.json
└── README.md
```

8.2 各ファイルの概要

ファイルパス	種類	説明
`app/layout.tsx`	レイアウト	アプリ全体の共通レイアウト。Header を含む
`app/page.tsx`	ページ	トップページ＝書籍一覧ページ（`/`）。Supabase からデータを取得して表示
`app/globals.css`	スタイル	Tailwind CSS のベーススタイル（`@import "tailwindcss";`）
`app/loading.tsx`	ページ	読み込み中に表示する画面（第7章）
`app/error.tsx`	ページ	エラーが起きたときの画面（第7章）
`app/books/new/page.tsx`	ページ	新規書籍追加ページ。BookForm を使用
`app/books/[id]/page.tsx`	ページ	書籍詳細ページ。動的ルートでIDに基づく表示
`app/books/[id]/not-found.tsx`	ページ	存在しない id を開いたときの案内画面（第8章）
`app/books/[id]/edit/page.tsx`	ページ	書籍編集ページ。既存データの更新
`components/Header.tsx`	コンポーネント	ヘッダーナビゲーション（第6章）
`components/BookCard.tsx`	コンポーネント	書籍カードの表示。一覧ページで使用
`components/BookList.tsx`	コンポーネント	書籍カードを並べて一覧にする（第7章）
`components/BookForm.tsx`	コンポーネント	書籍の追加・編集フォーム。新規追加と編集で共用
`components/DeleteButton.tsx`	コンポーネント	削除ボタン。確認ダイアログ付き
`components/StatusBadge.tsx`	コンポーネント	読書ステータス（読みたい・読書中・読了）のバッジ
`components/RatingStars.tsx`	コンポーネント	星評価の表示
`components/LoadingSpinner.tsx`	コンポーネント	読み込み中のくるくるアニメーション
`components/SearchBar.tsx`	コンポーネント	テキスト検索とステータスフィルタ（第8章）

ファイルパス	種類	説明
`components/SortSelect.tsx`	コンポーネント	並び替えの選択（第8章）
`lib/supabase/client.ts`	ユーティリティ	ブラウザ側（Client Component）用の Supabase クライアント
`lib/supabase/server.ts`	ユーティリティ	サーバー側（Server Component）用の Supabase クライアント
`app/api/health/route.ts`	API	ヘルスチェック（第10章 2.9.1 で追加した場合）
`scripts/check-env.mjs`	スクリプト	ビルド前の環境変数チェック（第10章 1.4.1 で追加した場合）
`types/book.ts`	型定義	Book 型の定義
`.env.local`	環境変数	Supabase の接続情報（Git管理外）
`next.config.ts`	設定	Next.js の設定ファイル（`cacheComponents` は入れない）
`eslint.config.mjs`	設定	ESLint の設定（`npm run lint` で使う）
`tsconfig.json`	設定	TypeScript のコンパイラ設定

練習問題（解答・解説つき）

この章のデプロイの内容を、問題で確かめましょう。まず自分で答えを考えてから、下の「解答と解説」を読んでください。

問題1: ○ と ƒ を読み取る（基本）

`npm run build` の最後に、次の表が表示されました。

コピー

```
Route (app)
┌ ƒ /
├ ○ /_not-found
├ ƒ /books/[id]
├ ƒ /books/[id]/edit
└ ○ /books/new
```

1. ビルドのときに HTML が作り置きされるページはどれですか。
2. 一覧ページの `/` が `ƒ` になっている理由を、一言で説明してください。

問題2: ビルドエラーを読む

`npm run build` で、次のエラーが出ました（Next.js 16.4.1 の実際の出力です）。

コピー

```
  Running TypeScript ...
components/BookCard.tsx(7,39): error TS2551: Property 'auther' does not exist on type 'Book'. Did you mean 'author'?
Failed to type check.
```

1. 直すべきファイルと場所はどこですか。
2. どう直せばよいですか。

問題3: 公開してはいけない環境変数

次の3つのうち、Vercel にこの名前で登録すると**危険なもの**はどれですか。理由も答えてください。

コピー

```
NEXT_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_xxxxxxxx
NEXT_PUBLIC_SUPABASE_SECRET_KEY=sb_secret_xxxxxxxx
```

問題4: 値を変えたのに反映されない

Vercel の Environment Variables で `NEXT_PUBLIC_SUPABASE_URL` の値を新しいプロジェクトのものに変えて「Save」しました。ところが本番サイトは、まだ古いプロジェクトのデータを表示しています。原因と、やるべきことを答えてください。

問題5: 友達にログイン画面が出る

新機能を `feature/review` ブランチに push し、Vercel が作った `book-management-app-git-feature-review-yourname.vercel.app` を友達に送ったところ、「Vercel のログイン画面が出て見られない」と言われました。なぜですか。また、完成した機能を友達に見てもらうには、どうするのがよいですか。

問題6: 本番にバグを出してしまった

Hobby プランで運用中、最新のデプロイ（v3）にバグがあり、一覧が表示されなくなりました。直前のデプロイ（v2）は正常でした。

1. 利用者への影響をいちばん早く止める方法は何ですか。
2. その方法を使ったあと、バグを直して `main` に push したのに、本番が新しくなりません。なぜですか。どうすれば元に戻りますか。

問題7: 1週間後にデータが消えた？

公開して10日ほどアプリを開かずにいたら、久しぶりに開いた本番サイトで本が1冊も表示されなくなっていました。コードは何も変えていません。最初に確かめるべきことは何ですか。

解答と解説

問題1の解答

1. `○` の付いた `/_not-found` と `/books/new` です。
2. サーバー用の Supabase クライアントが `cookies()` を使っていて、アクセスした人ごとに中身が変わるため、アクセスのたびにサーバーでページを作る（動的）からです。

解説: `○`（Static）は「ビルドのときに HTML を作り置きするページ」、`ƒ`（Dynamic）は「アクセスのたびにサーバーで作るページ」です。`/` が動的だからこそ、本を追加するとすぐ一覧に反映されます。`/books/[id]` と `/books/[id]/edit` は、URL の `[id]` ごとに中身が変わるので動的です（1.2「出力の読み方」）。

問題2の解答

1. `components/BookCard.tsx` の **7行目・39文字目**です。
2. `book.auther` を `book.author` に直します。

解説: `(7,39)` は「行, 列」、`error TS2551` は TypeScript の型エラーの種類です。`Property 'auther' does not exist on type 'Book'`（Book 型に auther というプロパティは無い）のあとに `Did you mean 'author'?`（author のこと？）とヒントが出ています。英語のつづり間違いはよくある原因なので、ヒントの単語と自分のコードを1文字ずつ見比べましょう（図10-6）。

問題3の解答

`NEXT_PUBLIC_SUPABASE_SECRET_KEY` が危険です。

解説: `NEXT_PUBLIC_` を付けた変数は、ビルドのときにブラウザ用の JS に値が書き込まれ、サイトを開いた人なら誰でも読めます（0.3 の実験1）。URL と Publishable key はもともと公開する前提の値で、データは RLS が守っているので問題ありません。一方、Secret key は RLS を素通りできる管理者用のキーなので、公開されるとデータベースを自由に読み書き・削除されてしまいます。秘密のキーには `NEXT_PUBLIC_` を付けず、もし公開してしまったら Supabase でキーを無効にして作り直します（3.1）。

問題4の解答

原因: 環境変数の変更は、すでにあるデプロイには反映されないからです。やること: Deployments で最新のデプロイの「…」→「Redeploy」を実行し、新しい値でビルドし直します。

解説: `NEXT_PUBLIC_` 付きの値は、ビルドの瞬間にブラウザ用の JS へ書き込まれます。値を変えても、ビルドし直すまでは古い値の JS が配られ続けます（0.3 の実験2）。さらに Vercel は、デプロイごとに環境変数の値を固定して保存しています。「Vercel で環境変数を変えたら Redeploy」とセットで覚えましょう（2.7）。

問題5の解答

理由: Deployment Protection の Standard Protection が有効で、プレビュー URL は Vercel にログインした権限のある人しか開けないからです。完成した機能は、Pull Request を `main` にマージして Production デプロイにし、本番用の短い URL（`book-management-app.vercel.app`）を送ります。

解説: 新しいプロジェクトでは、本番用の URL 以外（プレビュー URL や、ランダムな英数字の入った個別のデプロイ URL）が最初から保護されています。作りかけの画面が外部に見られないようにするための仕組みなので、学習中はオンのままが安全です。どうしてもプレビューを見せたいときは、共有用リンクを使います（2.6.2）。

問題6の解答

1. Vercel の Production Deployment の欄から **Instant Rollback** を実行し、v2 に戻します。ビルドしないので一瞬で切り替わります。
2. ロールバックすると「本番ドメインの自動の付け替え」が止まるため、push しても自動では本番に出ません。直した版が Ready になったら「**Undo Rollback**」（または Deployments の「…」→「Promote」）で本番にすると、いつもの自動デプロイに戻ります。

解説: Hobby プランで戻せるのは直前の本番デプロイだけですが、今回はそれ（v2）が正常なので問題ありません。焦ってコードを直すより、まず戻して被害を止めるのが鉄則です。Git の履歴もそろえたいときは、`git revert` で取り消しのコミットを作って push する方法もあります（2.10）。

問題7の解答

Supabase のダッシュボードを開き、プロジェクトが**一時停止**（**Pause**）していないかを確かめます。止まっていたら「Resume project」で再開します。

解説: Free プランのプロジェクトは、7日間ほとんど使われないと一時停止されることがあります。停止中は Vercel 側のアプリは動いているので、画面は表示されるのにデータだけが出ない、という症状になります。コードを直したり Redeploy したりしても解決しないので、データだけが出ないときは Supabase 側を先に疑いましょう（3.4）。

---

9. おわりに

この教材を完走したあなたへ

この教材を最後まで読み進め、書籍管理アプリを完成させたあなたに、心からの祝福を送ります。

プログラミングの学習は、決して簡単な道のりではありません。新しい概念が次々と登場し、エラーに悩まされ、「自分には向いていないのかも」と思うこともあったかもしれません。それでも、ここまでたどり着いたということは、あなたには確実に「学び続ける力」があるということです。

振り返ってみてください。この教材を始めた時点のあなたと、今のあなたは全く違います。

- **TypeScript** で型安全なコードが書けるようになりました

- **React** でインタラクティブな UI を構築できるようになりました
- **Next.js** でモダンな Web アプリケーションを作れるようになりました
- **Supabase** でデータベースを操作できるようになりました
- **Tailwind CSS** で美しいデザインを実装できるようになりました
- **Vercel** にデプロイして、世界中からアクセスできるアプリを公開できるようになりました

これらはすべて、現在のWeb開発の現場で実際に使われている技術です。あなたは、実務で通用するスキルの基礎をすでに手に入れています。

大切なこと

プログラミング学習で最も大切なのは、**手を動かし続けること**です。

- この教材のアプリに新しい機能を追加してみてください
- 別のアプリのアイデアを考えて、ゼロから作ってみてください
- エラーに出会ったら、それは学びのチャンスです
- わからないことがあれば、公式ドキュメントを読む習慣をつけてください
- 完璧を求めず、まず動くものを作り、少しずつ改善していくことが大切です

「できない」は「まだできない」

今の時点で理解しきれていないことがあっても、心配はいりません。「できない」のではなく、「まだできない」だけです。プロのエンジニアも、毎日新しいことを学び続けています。学習に終わりはありませんが、それこそがプログラミングの面白さでもあります。

あなたが作った書籍管理アプリは、あなたのポートフォリオの第一歩です。これからもっと多くのアプリを作り、スキルを磨いていってください。

フィードバック

この教材に関するご意見・ご感想・誤りの報告は、以下の方法でお寄せください。

- **GitHub Issues**: このリポジトリの Issues ページで報告
- **Pull Request**: 修正提案がある場合は PR を作成

皆さまのフィードバックが、この教材をより良いものにします。

お疲れさまでした。あなたの Web 開発の旅は、ここから始まります。